TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN



MÔN HỌC PHÂN TÍCH MÃ ĐỘC

BÁO CÁO CUỐI KÌ

PHÁT TRIỂN SCRIPT PYTHON TỰ ĐỘNG GIẢI MÃ CÁC CHUỖI STRINGS BỊ LÀM MỜ TRONG MALWARE BẰNG KỸ THUẬT EMULATION UNICORN

GVHD: Thầy Đinh Công Đoan

SVTH:

1. Huỳnh Thiên Hạo - 23162021
2. Trương Nguyễn Minh Hậu - 23162022
3. Võ Gia Huân - 23162027

TP. HỒ CHÍ MINH - NĂM 2026

MỤC LỤC

PHẦN 1: PHẦN MỞ ĐẦU 5

1.1. Tóm tắt ý tưởng 5

1.2. Đặt vấn đề 5

1.2.1. Tóm lược những nghiên cứu trong và ngoài nước liên quan đến đề tài 5

1.2.2. Tính cấp thiết cần nghiên cứu của đề tài 6

1.2.3. Một số tài liệu liên quan 6

1.2.4. Lý do chọn đề tài 7

1.2.5. Mục tiêu đề tài 7

1.2.6. Đối tượng và phạm vi nghiên cứu 8

1.2.7. Phương pháp nghiên cứu 8

1.2.8. Nội dung đề tài 9

PHẦN 2: NỘI DUNG 10

CHƯƠNG 1: GIỚI THIỆU 10

1.1. Tổng quan về malware và kỹ thuật làm mờ chuỗi (String Obfuscation) 10

1.2. Vai trò của việc phân tích mã hoá (deobfuscate string) trong malware 10

1.3. Giới thiệu kỹ thuật mô phỏng (emulation) trong malware 11

1.4. Tổng quan về Unicorn Engine trong phân tích malware 11

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU 13

2.1. Yêu cầu chức năng 13

2.1.1. Yêu cầu đầu vào 13

2.1.2. Yêu cầu xử lý 13

2.1.3. Yêu cầu đầu ra 14

2.2. Yêu cầu phi chức năng 14

2.2.1. Hiệu năng 14

2.2.2. Độ chính xác 15

2.2.3. Khả năng mở rộng 15

2.3. Sơ đồ nguyên lý hoạt động của hệ thống 15

2.4. Nguyên tắc làm việc của Script 16

2.4.1. Quy trình phân tích malware 16

2.4.2. Quy trình emulation 16

2.4.3. Quy trình trích xuất strings 16

2.5. Lựa chọn giải pháp 17
2.5.1. So sánh các phương pháp deobfuscate strings 17
2.5.2 Lý do chọn Unicorn Engine 17
CHƯƠNG 3: NHỮNG KIẾN THỨC LIÊN QUAN 18
3.1. Kỹ thuật làm mờ mã nguồn trong malware 18
3.1.1. XOR Encoding. 18
3.1.2. Base64 Encoding 19
3.1.3. Custom Encoding Algorithms (Thuật toán mã hóa tùy chỉnh) 21
3.1.4. Stack Strings 22
3.1.5. Encrypted Strings với Key từ Environment (Mã hóa chuỗi bằng khóa từ môi trường) 24
3.2. Công cụ Unicorn Engine 25
3.2.1. Giới thiệu về Unicorn Engine 25
3.2.2. Kiến trúc và cơ chế hoạt động 25
3.2.3. Các kiến trúc CPU được hỗ trợ 26
3.2.4. Cơ chế Hook, là nền tảng của kỹ thuật emulation phân tích 27
3.2.5. Ưu điểm của Unicorn Engine trong phân tích malware 28
3.2.6. Vị trí của Unicorn Engine trong hệ sinh thái công cụ bảo mật 28
3.2.7. Các API chính của Unicorn 29
3.3. Python và các thư viện hỗ trợ 29
3.3.1. Thư viện unicorn 30
3.3.2. Thư viện pefile 30
3.3.3. Thư viện capstone 31
3.3.4. Các thư viện khác 32
3.4. Các công cụ phân tích tĩnh 32
3.4.1. IDA Pro 33
3.4.2. Resource Hacker 34
3.4.3. Quy trình phối hợp giữa các công cụ phân tích tĩnh 35
3.5. Định dạng file PE 35
3.5.1. Cấu trúc tổng thể file PE 35
3.5.2. Các section trong file PE 36
3.5.3. Import Address Table (IAT) 37
3.5.4. Resource Section 38
3.5.5. Mối quan hệ giữa cấu trúc PE và quy trình emulation 39
3.6. Assembly và kiến trúc x86/x64 40

3.6.1. Các thanh ghi cơ bản 40
3.6.2. Các lệnh assembly thường gặp trong deobfuscation 41
3.6.3. Calling Conventions 42
3.6.4. Stack Frame 44
CHƯƠNG 4: ỨNG DỤNG 46
4.1. Mô tả tổng quan ứng dụng 46
4.1.1 Chức năng chính của script 46
4.1.2 Luồng hoạt động tổng thể 46
4.1.3. Các module chức năng chính 47
4.2. Sơ đồ quy trình làm việc 48
4.2.1 Kiến trúc tổng thể của Script 48
4.2.2. Sequence Diagram 49
4.2.3. Activity Diagram 49
4.3. Chi tiết các bước xây dựng ứng dụng 50
4.3.1 Module phân tích file PE 50
4.3.2. Module xác định hàm deobfuscation 50
4.3.3. Module thiết lập môi trường emulation 51
4.3.4. Module thực thi emulation 52
4.3.5. Module trích xuất và lưu trữ strings 52
4.4. Cài đặt và cấu hình 53
4.4.1. Yêu cầu hệ thống 53
4.4.2. Cài đặt dependecies 54
4.4.3. Cấu hình script 54
4.4.4. Hướng dẫn sử dụng 55
4.5. Demo và kết quả thử nghiệm 56
4.5.1. Kịch bản 1: Vượt qua kỹ thuật làm rối Control Flow 56
4.5.2. Kịch bản 2: Trích xuất Stack Strings ẩn 57
4.6. Phân tích những khó khăn và cách khắc phục 59
4.6.1. Khó khăn trong việc xác định hàm deobfuscation 59
4.6.2. Khó khăn trong emulation – Thiếu context 59
4.6.4. Khó khăn về hiệu năng 60
4.6.5. Khó khăn trong xử lý nhiều kiến trúc 60
PHẦN 3: KẾT LUẬN 61
5.1. Kết quả đạt được 61

5.2. Ưu điểm của đề tài 62
5.3. Nhược điểm và hạn chế 62
5.4. Hướng phát triển trong tương lai 62
TÀI LIỆU THAM KHẢO 64

PHẦN 1: PHẦN MỞ ĐẦU

1.1. Tóm tắt ý tưởng

Đề tài phát triển mã lệnh Python tự động giải mã chuỗi bị làm mờ (obfuscated strings) trong mã độc bằng kỹ thuật mô phỏng của Unicorn Engine. Quá trình này giúp trích xuất các chỉ số xâm phạm (IOC) và xác định hành vi mã độc, khắc phục tình trạng tốn thời gian và dễ sai sót của phương pháp phân tích thủ công trước các kỹ thuật che giấu phức tạp (XOR, stack strings, API hashing, mã hóa tùy chỉnh).

Quy trình hoạt động được tích hợp qua các giai đoạn: phân tích cấu trúc PE, xác định hàm giải mã, thiết lập môi trường mô phỏng, thực thi mã và trích xuất dữ liệu. Công cụ hỗ trợ kiến trúc vi xử lý x86 và x64, có khả năng xử lý nhiều kỹ thuật làm mờ, và được cấu trúc để nhúng trực tiếp vào các luồng phân tích tự động. Thử nghiệm trên mẫu thực tế cho thấy hệ thống trích xuất chuỗi với độ chính xác cao, rút ngắn thời gian xử lý, tạo cơ sở nền tảng cho các hệ thống phân tích mã độc tự động hóa.

1.2. Đặt vấn đề

1.2.1. Tóm lược những nghiên cứu trong và ngoài nước liên quan đến đề tài

Nghiên cứu quốc tế: Kỹ thuật mô phỏng (Emulation) thông qua Unicorn Engine và Python được ứng dụng trong các dự án như flare-emu (Mandiant) hay Trellix. Phương pháp này cho phép cô lập và thực thi hàm giải mã trên CPU ảo thay vì chạy toàn bộ mã độc trong hộp cát (sandbox), hỗ trợ trích xuất "stack strings" và vô hiệu hóa thuật toán mã hóa tùy chỉnh (XOR/AES) để khắc phục hạn chế của phân tích tĩnh.

Nghiên cứu trong nước: Các đơn vị an ninh mạng (Viettel Cyber Security, VNPT, BKAV) ứng dụng mô phỏng để trích xuất dữ liệu cấu hình (máy chủ C2, khóa mã hóa) từ mã độc phổ biến (Sodinokibi, Cobalt Strike). Hiện tại, các giải pháp này chủ yếu là công cụ nội bộ, chưa phát triển thành một framework có khả năng mở rộng.

Thách thức và giải pháp: Hạn chế cốt lõi của kỹ thuật mô phỏng là quá trình tự động khôi phục ngữ cảnh bộ nhớ (Context Recovery) và móc nối API (API Hooking), do lỗi thường phát sinh khi hàm giải mã tương tác trực tiếp với hệ điều hành. Đề tài ứng dụng công nghệ mô phỏng CPU

để tối ưu hóa quy trình nạp mã máy và tự động nhận diện tham số đầu vào, nhằm tăng tốc độ phân tích và giảm bớt các thao tác dịch ngược thủ công.

1.2.2. Tính cấp thiết cần nghiên cứu của đề tài

Theo thống kê từ AV-TEST Institute, số lượng mã độc mới phát sinh trên toàn cầu hiện ở mức hàng triệu mẫu. Dữ liệu của Kaspersky Security Network năm 2024 ghi nhận hơn 200.000 mẫu mã độc mới mỗi ngày, trong đó phần lớn tích hợp các kỹ thuật làm mờ (obfuscation) nhằm vô hiệu hóa công cụ nhận diện. Tốc độ gia tăng và mức độ phức tạp của mã độc hiện nay vượt quá giới hạn đáp ứng của phương pháp phân tích và dịch ngược thủ công.

Yêu cầu thực tế đòi hỏi các hệ thống phòng thủ phải trang bị năng lực phân tích và giải mã tự động. Việc nghiên cứu ứng dụng công nghệ mô phỏng (emulation) để tự động hóa trích xuất chuỗi (deobfuscate strings) giải quyết trực tiếp giới hạn này. Kết quả nghiên cứu cung cấp công cụ kỹ thuật phục vụ công tác ứng phó sự cố (Incident Response) và tình báo mối đe dọa (Threat Intelligence), đồng thời bổ sung cơ sở dữ liệu cho các nghiên cứu chuyên sâu về phân tích mã độc.

1.2.3. Một số tài liệu liên quan

- Nguyen Anh Quynh et al. (2015). "Unicorn: Next Generation CPU Emulator Framework". BlackHat USA 2015.
- Mandiant/FireEye. FLOSS - FireEye Labs Obfuscated String Solver. GitHub: mandiant/flare-floss.
- Sebastiaan Marinus van Wiltenburg. (2020). "Emulation-based deobfuscation of protected binaries". MSc Thesis, TU Delft.
- Sikorski, M., & Honig, A. (2012). Practical Malware Analysis. No Starch Press.
- Microsoft Security Intelligence. (2023). Microsoft Digital Defense Report 2023.
- Victor M. Alvarez. *YARA: The Pattern Matching Swiss Knife for Malware Researchers*. VirusTotal / Google.

- Google Security Research. *Tyrant / Binary Analysis Tools and Malware Research Papers*. Google Project Zero Blog.

- FLARE Team (FireEye / Mandiant). *CAPA – Detect Malware Capabilities Automatically*. GitHub: mandiant/capa.

1.2.4. Lý do chọn đề tài

Đề tài được triển khai dựa trên ba lý do chính:

Thứ nhất, về yêu cầu ứng dụng kỹ thuật: Phân tích mã độc và xử lý kỹ thuật obfuscation là các thao tác đòi hỏi phải có chuyên môn trong lĩnh vực an toàn thông tin. Quá trình xây dựng công cụ tự động hóa cung cấp phần mềm có khả năng tích hợp vào quá trình xử lý nghiệp vụ.

Thứ hai, về tính liên ngành: Việc thực hiện hệ thống yêu cầu vận dụng kết hợp các kiến thức chuyên môn thuộc các phân nhánh lập trình hệ thống, kiến trúc máy tính, phân tích nhị phân và an toàn mạng.

Thứ ba, về hiện trạng các tài liệu nghiên cứu: Dữ liệu, thông tin và công cụ chuyên ngành bằng tiếng Việt liên quan đến kỹ thuật mô phỏng giải mã chưa được phổ biến rộng rãi. Do đó, sản phẩm của đề tài sẽ góp phần bổ sung tài liệu tham khảo và công cụ chức năng cho hệ thống cơ sở dữ liệu chuyên ngành trong nước.

1.2.5. Mục tiêu đề tài

Đề tài được định hướng theo các mục tiêu kỹ thuật sau:

- Nghiên cứu và đồng thời làm rõ các kỹ thuật làm mờ chuỗi (string obfuscation) được áp dụng trong mã độc.

- Ứng dụng kiến trúc và giao diện lập trình (API) của Unicorn Engine vào quy trình mô phỏng mã nhị phân.
- Thiết lập mã Script Python để tự động hóa việc phát hiện, mô phỏng và trích xuất chuỗi từ tập tin định dạng PE thuộc kiến trúc x86 và x64.

- Thực thi kiểm thử mã lệnh trên các mẫu mã độc thực tế nhằm trích xuất dữ liệu kết quả và đo lường thông số hoạt động.
- Xây dựng cấu trúc tài liệu làm cơ sở cho quá trình triển khai, tái sử dụng và mở rộng mã nguồn. Nhằm người khác cũng có thể tái sử dụng và phát triển tiếp.

1.2.6. Đối tượng và phạm vi nghiên cứu

Nhóm lựa chọn đề tài này vì xuất phát từ nhu cầu thực tế trong lĩnh vực an ninh mạng hiện nay, khi mà các kỹ thuật che giấu mã (obfuscation) trong malware ngày càng được sử dụng phổ biến để tránh bị phát hiện và gây khó khăn cho quá trình phân tích. Việc tìm hiểu và xây dựng công cụ hỗ trợ tự động giải mã các chuỗi bị làm mờ giúp việc phân tích mã độc trở nên nhanh hơn và hiệu quả hơn.

Bên cạnh đó, đề tài mang lại giá trị học tập rõ rệt khi yêu cầu kết hợp nhiều kiến thức như lập trình Python, reverse engineering, kiến trúc máy tính và kỹ thuật giả lập (emulation). Thông qua quá trình thực hiện, nhóm có thể tiếp cận các kỹ thuật phân tích malware hiện đại và hiểu rõ hơn cách hoạt động bên trong của các công cụ như Unicorn Engine.

Ngoài ra, đây cũng là một chủ đề còn khá ít tài liệu và công cụ hỗ trợ bằng tiếng Việt. Vì vậy, việc thực hiện đề tài không chỉ giúp nhóm nâng cao kiến thức mà còn có thể đóng góp thêm tài liệu tham khảo cho cộng đồng sinh viên và người nghiên cứu an ninh mạng trong nước.

1.2.7. Phương pháp nghiên cứu

Phương pháp nghiên cứu được triển khai theo các quy trình kỹ thuật sau:

- Thu thập thông tin, dữ liệu liên quan: Tổng hợp và chắt lọc các thông tin từ các tài liệu học thuật, tài liệu kỹ thuật của Unicorn Engine và các báo cáo phân tích mã độc.
- Phân tích thực nghiệm: Áp dụng kỹ thuật dịch ngược (reverse engineering) thông qua phần mềm IDA Pro và Ghidra trên các tập tin mã độc để xác định và hiểu rõ cơ chế obfuscation.
- Thiết kế và lập trình: Xây dựng Scrypt theo mô hình module, kết hợp triển khai kiểm thử đơn vị (unit test) cho từng thành phần chức năng.

- Kiểm thử hệ thống: Vận hành phần mềm trên tập dữ liệu mẫu mã độc, thực hiện đo lường các chỉ số hiệu năng và độ chính xác của đầu ra.

- Hiệu chỉnh và cải tiến thuật toán: Phân tích dữ liệu từ các kết quả thất bại hoặc bị lỗi nhằm xác định nguyên nhân gây ra và tiến hành tối ưu hóa, cải thiện thuật toán.

1.2.8. Nội dung đề tài

Cấu trúc của báo cáo được phân chia thành ba phần chính, bao quát toàn bộ tiến trình nghiên cứu:

Phần mở đầu xác định bối cảnh nghiên cứu, diễn giải lý do lựa chọn đề tài, đồng thời vạch ra hệ thống mục tiêu và các phương pháp tiếp cận được áp dụng.

Phần nội dung gồm bốn chương trọng tâm:

- Chương 1: Cung cấp cái nhìn tổng quan về mã độc (malware) và các đặc tính của kỹ thuật mô phỏng (emulation).

- Chương 2: Đi sâu vào việc phân tích, xác định các yêu cầu chức năng và phi chức năng của hệ thống.

- Chương 3: Tổng hợp các kiến thức nền tảng làm cơ sở lý thuyết cho việc phát triển phần mềm.

- Chương 4: Diễn giải chi tiết quy trình xây dựng công cụ, thiết lập môi trường hoạt động và đánh giá các kết quả ứng dụng trên thực tế.

Phần kết luận tóm lược những thành quả kỹ thuật đã đạt được, ghi nhận các điểm hạn chế của hệ thống hiện tại, và đề xuất các định hướng tối ưu hóa cho giai đoạn phát triển tiếp theo.

PHẦN 2: NỘI DUNG

CHƯƠNG 1: GIỚI THIỆU

1.1. Tổng quan về malware và kỹ thuật làm mờ chuỗi (String Obfuscation)

Malware (malicious software) là thuật ngữ chỉ các loại phần mềm xâm nhập do tội phạm mạng phát triển, nhằm mục đích đánh cắp dữ liệu, gây tổn hại hoặc phá hủy máy tính và các hệ thống máy tính. Các loại mã độc phổ biến bao gồm virus, worm (sâu máy tính), Trojan, spyware (phần mềm gián điệp), adware (phần mềm quảng cáo) và ransomware (mã độc tống tiền). Trong các cuộc tấn công bằng mã độc gần đây, một lượng lớn dữ liệu đã bị trích xuất và rò rỉ trái phép.

String obfuscation là kỹ thuật làm mờ các chuỗi ký tự trong malware nhằm che giấu thông tin quan trọng và làm giảm khả năng bị phát hiện thông qua phân tích tĩnh. Thay vì lưu trữ trực tiếp các chuỗi dạng rõ ràng (plain text), malware sẽ biến đổi chúng thành các dạng mã hóa hoặc cấu trúc phức tạp và chỉ giải mã khi chương trình đang chạy (runtime).

1.2. Vai trò của việc phân tích mã hoá (deobfuscate string) trong malware

Deobfuscation là quá trình khôi phục lại các chuỗi hoặc đoạn mã đã bị làm mờ trong malware về dạng ban đầu. Đây là bước quan trọng trong phân tích mã độc nhằm hiểu rõ hành vi thực sự của chương trình.

Trong thực tế, malware thường sử dụng nhiều lớp obfuscation để che giấu thông tin quan trọng. Nếu không thực hiện deobfuscation, các dữ liệu như địa chỉ C2, API hệ thống, khóa mã hóa hoặc lệnh thực thi sẽ không thể được nhận diện chính xác, gây khó khăn cho quá trình phân tích.

Vai trò của deobfuscation bao gồm:

- Xác định hành vi của malware: giúp hiểu mục đích và chức năng của mã độc.
- Hỗ trợ phát hiện và phòng chống: tạo điều kiện xây dựng chữ ký và quy tắc phát hiện.
- Phục vụ điều tra an ninh mạng: truy vết nguồn gốc và cơ chế hoạt động của cuộc tấn công.

- Hỗ trợ reverse engineering: giảm độ phức tạp của mã, giúp phân tích dễ dàng hơn.

1.3. Giới thiệu kỹ thuật mô phỏng (emulation) trong malware

Emulation là kỹ thuật mô phỏng môi trường thực thi nhằm chạy và quan sát hành vi của malware trong một hệ thống an toàn. Thay vì phân tích mã nguồn trực tiếp, emulation cho phép thực thi malware trong môi trường giả lập để ghi nhận hành vi động của nó.

Khi chạy trong môi trường emulation, malware có thể thực hiện các hành vi như giải mã chuỗi, gọi API, thay đổi registry hoặc thiết lập kết nối mạng. Những hành vi này giúp nhà phân tích hiểu rõ cách thức hoạt động thực tế của mã độc.

Các môi trường emulation thường được triển khai thông qua máy ảo (virtual machine), sandbox hoặc các công cụ chuyên dụng trong phân tích malware. Những môi trường này giúp cô lập hệ thống thật khỏi nguy cơ lây nhiễm.

Ưu điểm của kỹ thuật emulation bao gồm:

- Quan sát hành vi thực tế của malware trong runtime.
- Thu thập dữ liệu đã bị mã hóa hoặc obfuscation.
- Giảm rủi ro cho hệ thống phân tích.
- Hỗ trợ tự động hóa quá trình phân tích.

Tuy nhiên, nhiều malware hiện đại có khả năng phát hiện môi trường ảo hoặc sandbox để né tránh phân tích, gây khó khăn cho việc nghiên cứu.

1.4. Tổng quan về Unicorn Engine trong phân tích malware

Unicorn Engine là framework emulation CPU mã nguồn mở hàng đầu, được phát triển bởi Nguyen Anh Quynh (tác giả QEMU) và ra mắt tại BlackHat USA 2015. Được tách ra từ user-mode core của QEMU, Unicorn loại bỏ toàn bộ OS emulation layer để tập trung vào CPU emulation thuần túy- nhẹ hơn và nhanh hơn đáng kể so với QEMU truyền thống.

Kiến trúc và tính năng cốt lõi

Unicorn sử dụng Tiny Code Generator (TCG)- JIT compiler của QEMU- để dịch instruction blocks từ guest architecture (x86, ARM...) sang host native code. Core workflow bao gồm: map memory regions (RWX permissions), set registers & initial PC, register hooks (code/mem access), và uc_emu_start(begin, until, timeout, count).

Case study: FLOSS (FireEye Labs Obfuscated String Solver) FLOSS sử dụng Unicorn để emulate 17 obfuscation algorithms bao gồm: XOR single/iterative/multi-byte keys, RC4 stream cipher, Base64 variants, ROT13 và Stack strings. Kết quả đạt accuracy 95%+ trên 10.000+ malware samples, nhanh hơn 1000x so với manual analys.

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU

2.1. Yêu cầu chức năng

Hệ thống (script phân tích malware) được xây dựng nhằm thực hiện quá trình phân tích và giải mã chuỗi bị obfuscation thông qua kỹ thuật emulation. Các yêu cầu chức năng được chia thành ba nhóm chính: đầu vào, xử lý và đầu ra.

2.1.1. Yêu cầu đầu vào

Script phải có khả năng nhận và xử lý các loại đầu vào sau:

- File PE (Portable Executable) định dạng .exe hoặc .dll chạy trên Windows, hỗ trợ cả kiến trúc 32-bit (x86) và 64-bit (x64).
- Đường Dẫn Đến File malware cần phân tích, được cung cấp thông qua tham số dòng lệnh, giúp hệ thống linh hoạt khi phân tích nhiều mẫu khác nhau.
- Địa chỉ hàm deobfuscation (có thể cấu hình tĩnh hoặc tự động phát hiện) cùng với offset so với ImageBase của file PE.
- Tùy chọn cấu hình: Bao gồm các tham số như thời gian giới hạn (timeout) cho emulation, kích thước bộ nhớ mô phỏng và chế độ debug nhằm hỗ trợ quá trình phân tích chi tiết hơn.

2.1.2. Yêu cầu xử lý

Hệ thống cần thực hiện các bước xử lý theo trình tự sau:

- Đọc và parse file PE: Sử dụng thư viện pefile để phân tích cấu trúc file, xác định các thông tin quan trọng như ImageBase, entry point, danh sách section và bảng IAT (Import Address Table).
- Thiết lập môi trường emulation: Khởi tạo Unicorn Engine với kiến trúc phù hợp (x86 hoặc x64). Thực hiện ánh xạ bộ nhớ (memory mapping) cho các vùng code, stack, heap và xây dựng fake IAT.

- Cấu hình API hooks: Tạo các hook handler để mô phỏng hành vi của các Windows API thường được malware sử dụng như GetModuleHandleA, FindResourceA, GlobalAlloc, LockResource, đảm bảo chương trình có thể chạy trong môi trường giả lập.
- Thực thi emulation: Chạy hàm deobfuscation trong môi trường kiểm soát với giới hạn thời gian và số lượng instruction nhằm tránh vòng lặp vô hạn hoặc hành vi bất thường.
- Trích xuất kết quả: Sau khi emulation hoàn tất, đọc dữ liệu từ vùng nhớ được chỉ định và áp dụng regex để lọc ra các chuỗi có ý nghĩa phục vụ phân tích.

2.1.3. Yêu cầu đầu ra

Hệ thống phải xuất ra các thông tin sau:

- Danh sách chuỗi đã giải mã (deobfuscated strings): Bao gồm các thông tin như địa chỉ C2 server, tên API, khóa mã hóa và các chuỗi cấu hình của malware.
- Thông tin debug: Ghi lại các Windows API được gọi trong quá trình emulation cùng với giá trị các thanh ghi quan trọng.
- Trạng thái thực thi: Xác định quá trình phân tích thành công hay thất bại, kèm thông báo lỗi chi tiết nếu có.
- Tùy chọn lưu kết quả: Cho phép xuất kết quả ra file .txt hoặc .json để phục vụ lưu trữ và phân tích sau này.

2.2. Yêu cầu phi chức năng

2.2.1. Hiệu năng

Script phải đảm bảo khả năng thực hiện quá trình emulation và trích xuất chuỗi (strings) trong thời gian tối đa không quá 30 giây đối với một hàm deobfuscation thông thường. Thời gian timeout mặc định cho mỗi lần emulation được thiết lập là 5 giây nhằm tránh tình trạng treo hoặc vòng lặp vô hạn.

Ngoài ra, mức sử dụng bộ nhớ của hệ thống không được vượt quá 512MB trong điều kiện hoạt động bình thường, đảm bảo khả năng chạy ổn định trên các môi trường máy tính phổ thông.

2.2.2. Độ chính xác

Hệ thống cần đảm bảo độ chính xác tối thiểu 85% trong việc trích xuất các chuỗi có ý nghĩa từ các mẫu malware thông thường. Tỷ lệ false positive (chuỗi không có ý nghĩa nhưng bị trích xuất) phải được giữ dưới 20% nhằm đảm bảo chất lượng kết quả phân tích.

2.2.3. Khả năng mở rộng

Script được thiết kế theo kiến trúc module hóa (modular architecture), cho phép dễ dàng mở rộng và tích hợp thêm các chức năng mới trong tương lai.

Hệ thống hỗ trợ:

- Bổ sung các Windows API hooks mới.
- Mở rộng hỗ trợ nhiều kiến trúc CPU như ARM, MIPS.
- Tích hợp thêm các định dạng file khác như ELF và Mach-O.
- Tách cấu hình như địa chỉ hàm deobfuscation và API hooks ra file cấu hình riêng để dễ bảo trì.
- Nhờ kiến trúc này, hệ thống có thể phát triển theo hướng một framework phân tích malware mở rộng phi chức năng.

2.3. Sơ đồ nguyên lý hoạt động của hệ thống

Hệ thống hoạt động theo quy trình tuyến tính từ đầu vào đến đầu ra, được chia thành 4 giai đoạn chính:

- Giai đoạn 1- Phân tích PE: Script sử dụng pefile để đọc header, xác định ImageBase, tìm kiếm resources, liệt kê IAT và tính toán địa chỉ thực của hàm deobfuscation.
- Giai đoạn 2- Thiết lập môi trường: Unicorn Engine được khởi tạo, các vùng bộ nhớ được cấp phát và ánh xạ, code sections được tải vào memory, Fake IAT được tạo để giả lập môi trường Windows.

- Giai đoạn 3- Emulation: Registers được thiết lập, hooks được cài đặt, hàm deobfuscation được thực thi từ địa chỉ entry point. Mọi lời gọi API được chặn bắt và xử lý bởi hook handlers.
- Giai đoạn 4- Trích xuất: Sau khi emulation kết thúc, script đọc nội dung từ vùng bộ nhớ kết quả (thường được trỏ bởi EAX), áp dụng Regex để lọc chuỗi ASCII có ý nghĩa và xuất kết quả.

2.4. Nguyên tắc làm việc của Script

2.4.1. Quy trình phân tích malware

Trước khi chạy script, nhà phân tích cần thực hiện bước phân tích tĩnh sơ bộ để xác định hàm deobfuscation mục tiêu. Sử dụng IDA Pro hoặc Ghidra để tìm các hàm có pattern đặc trưng: vòng lặp với phép XOR, truy cập Resource section, cấp phát heap động và copy chuỗi. Địa chỉ của hàm này sau đó được cung cấp cho script thông qua biến DECRYPT_FUNC.

2.4.2. Quy trình emulation

Script tải toàn bộ file PE vào bộ nhớ ảo của Unicorn Engine tại địa chỉ ImageBase chính xác như trong môi trường Windows thực. Điều này đảm bảo tất cả các địa chỉ tương đối (RVA) và absolute address đều hoạt động chính xác. Stack được tạo tại địa chỉ cố định 0x00100000 với kích thước 1MB. Return address được đặt là 0x00000000- khi hàm return về địa chỉ này, Unicorn sẽ dừng lại.

2.4.3. Quy trình trích xuất strings

Sau khi emulation hoàn thành, giá trị thanh ghi EAX thường chứa con trỏ đến bộ nhớ có chuỗi đã giải mã. Script đọc 256 bytes từ địa chỉ này và sử dụng biểu thức Regex [a-zA-Z0-9:/.\-_]{5,} để tìm các chuỗi ASCII có ý nghĩa. Ngưỡng tối thiểu 5 ký tự giúp loại bỏ noise trong khi vẫn capture được đủ thông tin.

2.5. Lựa chọn giải pháp

2.5.1. So sánh các phương pháp deobfuscate strings

Phương pháp	**Ưu điểm**	**Nhược điểm**	**Phù hợp**
Phân tích tĩnh	Nhanh an toàn	Kém hiệu quả với mã hóa phức tạp	Malware đơn giản
Dynamic analysis	Hiệu quả cao	Nguy hiểm, cần sandbox đắt tiền	Malware bất kỳ
Scripted unpacking	Nhanh	Chỉ với packer đã biết	Packer malware
CPUEmulation (chọn)	An toàn và hiệu quả	Phức tạp khi thiếu context	Mọi loại malware

2.5.2 Lý do chọn Unicorn Engine

Unicorn Engine được lựa chọn vì những ưu điểm vượt trội:

(1) Mã nguồn mở và miễn phí.

(2) Hỗ trợ nhiều kiến trúc CPU (x86, x64, ARM, MIPS, SPARC).

(3) Binding Python chính thức cho phép tích hợp dễ dàng.

(4) Hiệu năng cao nhờ dựa trên QEMU.

(5) Kiểm soát chi tiết từng instruction.

(6) Cộng đồng lớn và tài liệu phong phú.

(7) Đã được sử dụng rộng rãi trong cộng đồng malware analysis.

CHƯƠNG 3: NHỮNG KIẾN THỨC LIÊN QUAN

3.1. Kỹ thuật làm mờ mã nguồn trong malware

Kỹ thuật làm mờ chuỗi (string obfuscation) được kẻ tấn công sử dụng để né tránh sự phát hiện của các công cụ bảo mật và phần mềm diệt virus. Thay vì lưu trữ chuỗi dưới dạng văn bản rõ ràng (plaintext) dễ bị các công cụ như strings hay IDA Pro phát hiện, mã độc áp dụng các kỹ thuật biến đổi khác nhau. Dựa trên tài liệu, dưới đây là các kỹ thuật cụ thể:

3.1.1. XOR Encoding.

Là phương pháp phổ biến nhất nhờ tính đơn giản và hiệu quả. Cơ chế: Mỗi byte của chuỗi gốc được thực hiện phép toán XOR với một khóa (key) cố định hoặc biến đổi. Quá trình giải mã được thực hiện bằng cách XOR lại với cùng khóa đó. Đặc điểm: Tốc độ xử lý nhanh và không yêu cầu khóa có kích thước lớn. Tuy nhiên, nhược điểm là dễ bị các chuyên gia phát hiện thông qua kỹ thuật phân tích tần suất (frequency analysis).

Ví dụ:

```
section .data
    encrypted_str db 0x4E, 0x5A, 0x56, 0x14, 0x4B, 0x5C, 0x41
    xor_key       equ 0x2A


attack_loop:
    movzx eax, byte [esi]
    inc   dword [freq + eax*4]  ; đếm tần suất từng byte
    inc   esi
    loop  attack_loop           ; lặp hết ciphertext

; trong tiếng Anh, space (0x20) xuất hiện nhiều nhất
; => peak_byte XOR 0x20 = key thật
```

```
    xor   edx, 0x20            ; edx = key đoán được

decrypt_loop:
    movzx eax, byte [esi]      ; load byte mã hóa
    xor   eax, edx             ; XOR với key đoán được
    mov   byte [edi], al       ; lưu byte đã giải mã
    inc   esi                  ; next source
    inc   edi                  ; next destination
    loop  decrypt_loop         ; lặp lại
```

3.1.2. Base64 Encoding

Mã độc sử dụng Base64 để che giấu nội dung chuỗi gốc. Cơ chế: Mặc dù không phải là thuật toán mã hóa thực sự do không sử dụng khóa bí mật, Base64 làm chuỗi trở nên khó đọc, giúp qua mặt các quy tắc phát hiện cơ bản. Đặc điểm: Kỹ thuật này thường không đứng độc lập mà được kết hợp với phép toán XOR hoặc các lớp mã hóa bổ sung khác nhằm gia tăng độ phức tạp. Ví dụ:

```
section .data
    ; "Hello" đã được Base64 encode = "SGVsbG8="
    encoded_str db 'S','G','V','s','b','G','8','=', 0

    ; bảng Base64 alphabet
    b64_table db
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"

section .bss
    decoded resb 64
```

```
decode_loop:
    ; load 4 ký tự Base64
    movzx eax, byte [esi]       ; ký tự 1
    movzx ebx, byte [esi+1]     ; ký tự 2
    movzx ecx, byte [esi+2]     ; ký tự 3
    movzx edx, byte [esi+3]     ; ký tự 4

    ; tra bảng → lấy giá trị 6-bit
    call  lookup_b64            ; eax = index trong b64_table

    ; ghép 4x6-bit = 3 byte output
    shl   eax, 18               ; byte1: bits 23-18
    shl   ebx, 12               ; byte1: bits 17-12
    shl   ecx, 6                ; byte2: bits 11-6
    ; edx giữ nguyên            ; byte3: bits 5-0
    or    eax, ebx
    or    eax, ecx
    or    eax, edx              ; eax = 3 byte gộp lại

    ; tách và lưu 3 byte
    mov   byte [edi],   ah      ; byte 1
    shr   eax, 8
    mov   byte [edi+1], ah      ; byte 2
    shr   eax, 8
    mov   byte [edi+2], al      ; byte 3
```

```
    add   esi, 4              ; next 4 ký tự Base64
    add   edi, 3              ; next 3 byte output
    loop  decode_loop
```

3.1.3. Custom Encoding Algorithms (Thuật toán mã hóa tùy chỉnh)

Thường được ứng dụng bởi các mối đe dọa an ninh cấp độ cao (APT). Cơ chế: Kẻ tấn công tự triển khai lại các biến thể từ các thuật toán tiêu chuẩn như Caesar, Vigenere, RC4 hoặc AES với một số thay đổi về logic. Đặc điểm: Đây là loại che giấu khó đối phó nhất trong phân tích tĩnh vì các công cụ không thể tự động nhận diện dựa trên cơ chế so khớp mẫu (pattern matching) thông thường.

```
section .data
    ; Vigenere biến thể: key xoay + Caesar shift thay đổi theo vị trí
    encrypted_str db 0x56, 0x42, 0x71, 0x38, 0x6C, 0x29, 0x4D, 0
    vig_key       db 0x13, 0x07, 0x2B, 0x1F, 0      ; key 4 byte, xoay vòng
    key_len       equ 4


decode_loop:
    movzx eax, byte [esi]          ; load byte mã hóa
    movzx ebx, byte [edi_key]      ; load byte key hiện tại

    ; biến thể 1: XOR với key
    xor   eax, ebx                 ; giải XOR cơ bản

    ; biến thể 2: Caesar shift ngược theo vị trí
    mov   ecx, [pos]               ; ecx = vị trí hiện tại
    and   ecx, 0x03                ; mod 4 (shift = 0,1,2,3 xoay vòng)
```

```
    sub   eax, ecx              ; trừ shift (Caesar ngược)
    and   eax, 0xFF             ; giữ trong 1 byte

    ; biến thể 3: rotate right theo pos
    mov   cl, [pos]
    and   cl, 0x07              ; rotate 0-7 bit
    ror   al, cl                ; xoay bit ngược lại

    mov   byte [out_buf + ebp], al  ; lưu byte đã giải

    ; cập nhật con trỏ key (xoay vòng)
    inc   edi_key
    mov   ecx, vig_key
    add   ecx, key_len
    cmp   edi_key, ecx
    jl    .no_reset
    lea   edi_key, [vig_key]        ; reset về đầu key
.no_reset:

    inc   dword [pos]           ; tăng vị trí
    inc   esi                   ; next byte
    loop  decode_loop
```

3.1.4. Stack Strings

Phương pháp này chia nhỏ chuỗi để tránh việc lưu trữ liền mạch. Cơ chế: Mã độc tiến hành xây dựng chuỗi từng byte một trực tiếp trên bộ nhớ stack tại thời điểm thực thi (runtime) thay vì

lưu trữ toàn bộ chuỗi hoàn chỉnh trong phân vùng dữ liệu (data section). Đặc điểm: Do mỗi byte được đẩy lên stack thông qua một lệnh mov hoàn toàn độc lập, các công cụ phân tích tĩnh bị vô hiệu hóa trong việc trích xuất ra một chuỗi đầy đủ.

```
; biến thể 1: dùng giá trị hex thay ký tự trực tiếp
mov byte ptr [ebp-7], 0x63      ; 'c'
mov byte ptr [ebp-6], 0x6D      ; 'm'
mov byte ptr [ebp-5], 0x64      ; 'd'
mov byte ptr [ebp-4], 0x2E      ; '.'
mov byte ptr [ebp-3], 0x65      ; 'e'
mov byte ptr [ebp-2], 0x78      ; 'x'
mov byte ptr [ebp-1], 0x65      ; 'e'
mov byte ptr [ebp-0], 0x00      ; null terminator

; biến thể 2: push dword (4 byte một lần) => ít lệnh hơn, khó đọc hơn
; "cmd." = 0x2E646D63, "exe\0" = 0x00657865
push 0x00657865                 ; đẩy "exe\0" lên stack trước
push 0x2E646D63                 ; đẩy "cmd." lên stack sau
mov  eax, esp                   ; eax trỏ tới "cmd.exe\0"

; biến thể 3: XOR từng byte trước khi ghi => thêm 1 lớp obfuscation
mov  al, 0x48                   ; 'c' XOR 0x2B = 0x48
xor  al, 0x2B                   ; giải ra 0x63 = 'c'
mov  byte ptr [ebp-7], al

mov  al, 0x46                   ; 'm' XOR 0x2B = 0x46
xor  al, 0x2B                   ; giải ra 0x6D = 'm'
```

```
mov  byte ptr [ebp-6], al

mov  al, 0x4F                ; 'd' XOR 0x2B = 0x4F
xor  al, 0x2B                ; giải ra 0x64 = 'd'
mov  byte ptr [ebp-5], al
; ... tiếp tục cho các byte còn lại

lea  eax, [ebp-7]            ; eax = địa chỉ chuỗi "cmd.exe"
push eax                     ; truyền vào WinExec/CreateProcess
```

3.1.5. Encrypted Strings với Key từ Environment (Mã hóa chuỗi bằng khóa từ môi trường)

Đây là kỹ thuật gắn chặt quá trình giải mã vào môi trường thực thi của nạn nhân. Cơ chế: Mã độc thu thập các thông số đặc thù từ hệ thống (như hostname, username, volume serial number) để làm khóa (key) giải mã cho các chuỗi. Đặc điểm: Kỹ thuật này trực tiếp đánh bại các phương pháp mô phỏng tĩnh (static emulation), bởi mã độc sẽ không thể tự giải mã nếu không nhận được chính xác các giá trị môi trường như trên máy nạn nhân.

```
Dấu hiệu đặc trưng nhất của kĩ thuật này:
GetComputerName / GetVolumeInfo
        ↓
    hàm hash (MD5/custom)
        ↓
    decrypt loop (XOR/RC4)
        ↓
    chuỗi hệ thống (URL, registry key...)
```

3.2. Công cụ Unicorn Engine

3.2.1. Giới thiệu về Unicorn Engine

Unicorn Engine là một framework giả lập CPU (CPU emulation framework) mã nguồn mở, được phát triển bởi Nguyen Anh Quynh, Dang Hoang Vu và các cộng sự vào năm 2015, công bố lần đầu tại hội nghị bảo mật BlackHat USA 2015. Framework được xây dựng trên cơ sở kế thừa và tái cấu trúc thành phần dịch mã động (Tiny Code Generator - TCG) của QEMU, nhưng được thiết kế lại theo hướng nhúng gọn (embeddable), tập trung hoàn toàn vào việc giả lập CPU mà không mang theo phần hệ điều hành hay thiết bị ngoại vi.

Khác với các giải pháp giả lập hệ thống đầy đủ như QEMU hay VirtualBox, Unicorn Engine hoạt động ở cấp độ CPU thuần túy: framework chỉ mô phỏng các thanh ghi (registers), không gian bộ nhớ và luồng thực thi lệnh máy, mà không yêu cầu kernel, hệ điều hành hay phần cứng thật. Đặc điểm này làm cho Unicorn trở thành công cụ lý tưởng cho các tác vụ phân tích mã nhị phân (binary analysis) trong lĩnh vực bảo mật thông tin.

Unicorn Engine được phát hành theo giấy phép LGPL2, hiện được duy trình tích cực trên GitHub và được sử dụng rộng rãi trong cộng đồng nghiên cứu bảo mật toàn cầu.

3.2.2. Kiến trúc và cơ chế hoạt động

Unicorn Engine được tổ chức theo mô hình ba tầng rõ ràng, đảm bảo tính linh hoạt và khả năng tích hợp cao:

Tầng binding ngôn ngữ là lớp ngoài cùng, cung cấp giao diện lập trình cho hơn 15 ngôn ngữ bao gồm Python, C/C++, Go, Rust, Java, .NET, Ruby, Haskell và nhiều ngôn ngữ khác. Trong đề tài này, binding Python được sử dụng do tính phổ biến và hệ sinh thái phong phú trong lĩnh vực phân tích bảo mật.

Tầng Core API là trung tâm của framework, cung cấp tập hàm C thống nhất. Các hàm cốt lõi bao gồm:

uc_open() - khởi tạo instance giả lập với kiến trúc và chế độ CPU được chỉ định

uc_mem_map() - ánh xạ vùng bộ nhớ ảo

uc_mem_write() / uc_mem_read() - ghi và đọc dữ liệu vào bộ nhớ ảo

uc_reg_write() / uc_reg_read() - thao tác với các thanh ghi CPU

uc_hook_add() - đăng ký hàm callback (hook) theo dõi quá trình thực thi

uc_emu_start() - bắt đầu quá trình giả lập từ địa chỉ chỉ định

Tầng QEMU TCG backend là lớp dưới cùng, thực hiện công việc dịch mã động: chuyển đổi tập lệnh của kiến trúc CPU gốc (guest) thành mã native trên máy chủ (host). Cơ chế dịch theo khối cơ bản (basic block translation) giúp Unicorn đạt tốc độ giả lập đủ cao cho các tác vụ phân tích thực tế.

3.2.3. Các kiến trúc CPU được hỗ trợ

Unicorn Engine hỗ trợ đa dạng kiến trúc CPU, bao gồm:

Kiến trúc	**Phiên bản hỗ trợ**
x86 / x86-64	16-bit, 32-bit, 64-bit
ARM	ARM, Thumb, Thumb-2
ARM64 (AArch64)	64-bit
MIPS	MIPS32, MIPS64, MicroMIPS

RISC-V	RV32, RV64
SPARC	SPARC32, SPARC64
M68K	32-bit
S390X	64-bit

Trong phạm vi đề tài này, trọng tâm giả lập là kiến trúc x86/x86-64, vốn là nền tảng phổ biến nhất của các mẫu malware trên hệ điều hành Windows.

3.2.4. Cơ chế Hook, là nền tảng của kỹ thuật emulation phân tích

Hệ thống hook (hook callback) là tính năng quan trọng nhất của Unicorn Engine đối với bài toán phân tích mã độc. Hook cho phép người dùng đăng ký các hàm callback được tự động gọi tại các sự kiện cụ thể trong quá trình giả lập, mà không cần can thiệp vào luồng thực thi.

Các loại hook chính được sử dụng trong đề tài:

- UC_HOOK_CODE - kích hoạt trước mỗi lệnh máy được thực thi. Cho phép ghi lại (trace) toàn bộ luồng thực thi, kiểm tra giá trị thanh ghi tại từng bước, hoặc dừng giả lập khi đạt điều kiện mong muốn.
- UC_HOOK_MEM_READ / UC_HOOK_MEM_WRITE - kích hoạt khi có thao tác đọc hoặc ghi bộ nhớ. Cho phép theo dõi các vùng bộ nhớ chứa chuỗi đã được giải mã, từ đó thu thập kết quả ngay sau khi malware tự thực hiện xử lý.
- UC_HOOK_MEM_INVALID - kích hoạt khi có truy cập bộ nhớ không hợp lệ. Được dùng để xử lý lỗi và ngăn chặn giả lập bị crash do mã độc cố tình truy cập địa chỉ nằm ngoài vùng được ánh xạ.

Cơ chế hook là yếu tố cho phép script Python trong đề tài quan sát và thu thập kết quả giải mã chuỗi một cách tự động, thay thế hoàn toàn cho việc phân tích thủ công bằng debugger.

3.2.5. Ưu điểm của Unicorn Engine trong phân tích malware

So với các phương pháp phân tích truyền thống, Unicorn Engine mang lại một số ưu điểm đặc biệt phù hợp với bài toán giải mã chuỗi bị làm mờ (string deobfuscation):

- An toàn và cách ly hoàn toàn: Mã độc được thực thi trong môi trường giả lập hoàn toàn tách biệt khỏi hệ điều hành thật. Malware không thể tạo tiến trình, kết nối mạng, hay ghi file ra ngoài phạm vi vùng nhớ được kiểm soát. Đây là ưu điểm cốt lõi so với phương pháp chạy trực tiếp trong sandbox hệ điều hành.

- Kiểm soát chi tiết đến từng lệnh: Người phân tích có thể dừng, quan sát, và thậm chí sửa đổi trạng thái CPU sau bất kỳ lệnh nào. Mức độ kiểm soát mà ngay cả các debugger tiên tiến cũng khó đạt được trong môi trường tự động hóa.

- Không phụ thuộc hệ điều hành: Các đoạn mã giải mã của malware thường không gọi Windows API mà chỉ thao tác thuần CPU (XOR, ADD, ROR, MOV...). Unicorn có thể thực thi chính xác các đoạn mã này mà không cần giả lập toàn bộ môi trường Windows.

- Khả năng tự động hóa cao: Toàn bộ quy trình - nạp mã, thiết lập ngữ cảnh, thực thi, thu thập kết quả - có thể được viết thành script Python tái sử dụng, cho phép xử lý hàng loạt mẫu malware mà không cần can thiệp thủ công.

- Tốc độ phân tích: So với phân tích tĩnh (cần reverse toàn bộ thuật toán) hay phân tích động qua debugger (cần thao tác thủ công), emulation có thể xử lý toàn bộ bảng chuỗi bị làm mờ trong vài giây.

3.2.6. Vị trí của Unicorn Engine trong hệ sinh thái công cụ bảo mật

Unicorn Engine không tồn tại độc lập mà là thành phần nền tảng (foundation layer) cho nhiều framework phân tích bảo mật cấp cao hơn. Hai dự án tiêu biểu xây dựng trực tiếp trên Unicorn là Angr (framework phân tích nhị phân và thực thi ký hiệu (symbolic execution)) và Qiling Framework (framework giả lập hệ thống đầy đủ hơn, bổ sung lớp giả lập hệ điều hành (Windows, Linux, macOS) lên trên nhân Unicorn).

Trong đề tài này, Unicorn được sử dụng trực tiếp ở tầng thấp nhất, được tận dụng tối đa hiệu năng và tính kiểm soát của framework, thay vì thông qua các lớp trừu tượng cấp cao. Cách tiếp cận này phù hợp với bài toán giải mã chuỗi vốn không đòi hỏi giả lập hệ điều hành nhưng yêu cầu độ chính xác và tốc độ cao trong việc thực thi các vòng lặp mã hóa nhỏ.

3.2.7. Các API chính của Unicorn

API	Mô tả	Ví dụ
Uc(arch, mode)	Khởi tạo engine	Uc(UC_ARCH_X86, UC_MODE_32)
mem_map(addr, size, perms)	Cấp phát vùng nhớ	uc.mem_map(0x1000, 0x1000, UC_PROT_ALL)
mem_write(addr, data)	Ghi dữ liệu vào memory	uc.mem_write(0x1000, b'\x90')
mem_read(addr, size)	Đọc dữ liệu từ memory	data = uc.mem_read(0x1000, 256)
reg_write(regid, value)	Ghi giá trị thanh ghi	uc.reg_write(UC_X86_REG_EAX, 0)
reg_read(regid)	Đọc giá trị thanh ghi	eax = uc.reg_read(UC_X86_REG_EAX)
hook_add(type, callback)	Thêm callback hook	uc.hook_add(UC_HOOK_CODE, cb)
emu_start(begin, until)	Bắt đầu emulation	uc.emu_start(0x1000, 0x2000)

3.3. Python và các thư viện hỗ trợ

Python là ngôn ngữ lập trình được lựa chọn chính trong đề tài này nhờ hệ sinh thái thư viện phong phú dành cho phân tích bảo mật, cú pháp ngắn gọn và khả năng tích hợp liền mạch với các công cụ như Unicorn Engine. Các thư viện được sử dụng trong script có thể chia thành hai nhóm: thư viện chuyên biệt cho emulation và phân tích PE còn lại là các thư viện tiện ích hỗ trợ xử lý dữ liệu nhị phân.

3.3.1. Thư viện unicorn

Thư viện Python unicorn là binding chính thức của Unicorn Engine dành cho ngôn ngữ Python, được duy trì bởi nhóm phát triển gốc và đảm bảo tương thích hoàn toàn với C API. Cài đặt thư viện đơn giản thông qua pip: *pip install unicorn*

Thư viện cung cấp đầy đủ các thành phần cần thiết để xây dựng môi trường giả lập:

- Class Uc: Lớp trung tâm đại diện cho một instance giả lập CPU. Mỗi đối tượng Uc được khởi tạo với kiến trúc và chế độ CPU cụ thể, sau đó được sử dụng để thao tác bộ nhớ, thanh ghi và điều khiển quá trình thực thi.
- Các hằng số UC_ARCH_* và UC_MODE_*: Xác định kiến trúc CPU (x86, ARM, MIPS...) và chế độ hoạt động (32-bit, 64-bit, Thumb...) khi khởi tạo engine.
- Các hằng số UC_HOOK_*: Xác định loại sự kiện mà callback sẽ được kích hoạt, bao gồm UC_HOOK_CODE (theo dõi từng lệnh), UC_HOOK_MEM_READ/WRITE (theo dõi truy cập bộ nhớ), và UC_HOOK_MEM_INVALID (xử lý truy cập bộ nhớ không hợp lệ).
- Exception class UcError: Lớp ngoại lệ cho phép bắt và xử lý các lỗi phát sinh trong quá trình giả lập như lỗi ánh xạ bộ nhớ, lỗi thực thi lệnh không hợp lệ hay lỗi truy cập ngoài vùng nhớ.

3.3.2. Thư viện pefile

Pefile là thư viện Python chuyên dụng để parse và phân tích cú pháp file PE (Portable Executable), định dạng file thực thi chuẩn trên hệ điều hành Windows ngoài ra còn có thể áp dụng cho các file .exe, .dll, .sys và nhiều định dạng khác. Cài đặt thông qua: *pip install pefile*

Thư viện cho phép truy cập toàn diện vào mọi thành phần cấu trúc của file PE mà không cần phải tự xử lý thủ công các offset nhị phân, bao gồm: DOS header, NT header, Optional header, section headers, import/export tables, resource directory, relocation table và các data directories khác.

Trong script của đề tài, pefile đảm nhận các nhiệm vụ thiết yếu:

- Đọc ImageBase và thông tin header: Lấy địa chỉ nạp mặc định của PE file, kích thước các section, và các thông số cần thiết để tái tạo không gian bộ nhớ ảo trung thực cho quá trình emulation.
- Lấy danh sách sections và raw data: Nạp từng section (.text, .data, .rdata,...) vào đúng địa chỉ ảo tương ứng trong môi trường Unicorn.
- Tìm và trích xuất Resource 101: Định vị payload đã bị mã hóa được nhúng trong resource section của malware.
- Xây dựng fake IAT (Import Address Table): Ánh xạ các địa chỉ hàm thư viện giả để malware có thể tham chiếu mà không gây lỗi trong quá trình giả lập.

3.3.3. Thư viện capstone

Capstone là thư viện disassembly đa kiến trúc hiệu năng cao, hỗ trợ hơn 20 kiến trúc CPU bao gồm x86/x64, ARM, MIPS, RISC-V và nhiều kiến trúc khác. Thư viện được thiết kế để tích hợp tự nhiên với Unicorn Engine, tạo thành cặp công cụ emulation.

Cài đặt thông qua: *pip install capstone*

Trong ngữ cảnh của đề tài, Capstone được tích hợp vào bên trong hook callback UC_HOOK_CODE để thực hiện disassembly theo thời gian thực trong quá trình emulation. Cụ thể, tại mỗi lệnh máy được thực thi, hook callback sẽ đọc byte mã máy tại địa chỉ hiện tại, truyền vào Capstone để nhận về mnemonic và operand tương ứng, sau đó ghi ra log theo dạng:

```
0x00401050  xor   ecx, ecx
0x00401052  mov   byte ptr [eax], cl
0x00401054  inc   eax
```

Tính năng này rất hữu ích trong giai đoạn debug script, cho phép người phân tích xác minh rằng luồng thực thi đang đi đúng hướng và các lệnh giải mã đang hoạt động như kỳ vọng trước khi thu thập kết quả cuối cùng.

3.3.4. Các thư viện khác

Ngoài ba thư viện chính kể trên, script còn sử dụng một số thư viện tiện ích để xử lý dữ liệu nhị phân và tương tác với môi trường thực thi.

Thư viện	**Chức năng**	**Ứng dụng trong đề tài**
struct	Pack/unpack dữ liệu nhị phân theo định dạng	Đọc và ghi các giá trị 32/64-bit vào vùng nhớ ảo của Unicorn
re	Tìm kiếm pattern bằng biểu thức chính quy	Trích xuất chuỗi printable từ vùng nhớ sau khi emulation hoàn tất
sys	Tương tác với môi trường hệ thống Python	Đọc tham số dòng lệnh (đường dẫn file mẫu malware)
os	Thao tác với hệ thống file và đường dẫn	Kiểm tra sự tồn tại của file, xử lý đường dẫn đa nền tảng
lief	Phân tích định dạng binary đa nền tảng	Phân tích PE/ELF/Mach-O nâng cao khi pefile không đủ
capstone	Disassembly engine đa kiến trúc	Debug và theo dõi luồng thực thi trong quá trình emulation

3.4. Các công cụ phân tích tĩnh

Phân tích tĩnh (static analysis) là bước đầu tiên và không thể thiếu trong quy trình phân tích malware, cho phép nghiên cứu cấu trúc và hành vi của mẫu mà không cần thực thi trực tiếp. Trong đề tài này, hai công cụ phân tích tĩnh chính được sử dụng là IDA Pro và Resource Hacker, mỗi công cụ đảm nhận một vai trò riêng biệt trong giai đoạn tiền xử lý trước khi tiến hành emulation.

3.4.1. IDA Pro

IDA Pro (Interactive DisAssembler) do hãng Hex-Rays phát triển là công cụ phân tích nhị phân chuyên nghiệp hàng đầu trong ngành bảo mật thông tin, được sử dụng rộng rãi bởi các nhà nghiên cứu bảo mật, reverse engineer và các tổ chức an ninh mạng toàn cầu. Kể từ khi ra mắt vào đầu những năm 1990, IDA Pro liên tục được phát triển và hiện hỗ trợ hơn 100 kiến trúc processor cùng hàng chục định dạng file thực thi khác nhau.

Tính năng cốt lõi:

IDA Pro tích hợp hai thành phần chính trong một môi trường làm việc thống nhất. Disassembler thực hiện chuyển đổi mã máy nhị phân thành hợp ngữ (assembly), đồng thời tự động nhận diện hàm, xây dựng đồ thị luồng điều khiển (Control Flow Graph - CFG) và phục hồi tên các hàm thư viện thông qua cơ sở dữ liệu chữ ký FLIRT. Decompiler (Hex-Rays Decompiler) tiến thêm một bước, chuyển đổi mã assembly thành mã giả C (pseudocode) ở mức độ đọc được, giúp rút ngắn đáng kể thời gian phân tích so với việc đọc thuần hợp ngữ.

Ngoài ra, IDA Pro hỗ trợ hệ thống plugin phong phú thông qua IDAPython cho phép viết script Python để tự động hóa các tác vụ phân tích như đặt comment hàng loạt, tìm kiếm pattern, hoặc trích xuất thông tin có cấu trúc từ file nhị phân.

Vai trò trong đề tài:

Trong đề tài này, IDA Pro được sử dụng trong giai đoạn phân tích sơ bộ mẫu Lab13-01.exe nhằm phục vụ các mục tiêu sau:

Xác định hàm deobfuscation: Thông qua phân tích đồ thị CFG và pseudocode, hàm thực hiện giải mã chuỗi được xác định tại offset 0x1300 so với ImageBase. Đây là thông tin đầu vào quan trọng để script Python biết chính xác địa chỉ bắt đầu (emu_start) và kết thúc (emu_until) khi gọi Unicorn Engine.

Xác định convention gọi hàm: Phân tích cách malware truyền tham số vào hàm giải mã (thông qua stack hay thanh ghi) để script có thể thiết lập đúng trạng thái CPU trước khi emulation.

Nhận diện cấu trúc dữ liệu: Xác định vị trí bảng chuỗi bị làm mờ trong các section, định dạng lưu trữ (độ dài, con trỏ, key) để script có thể lặp qua toàn bộ danh sách một cách tự động.

Kiểm tra kết quả: Sau khi script trả về chuỗi đã giải mã, IDA Pro được dùng để đối chiếu kết quả và đặt comment trực tiếp vào database IDA tại các vị trí sử dụng chuỗi đó trong mã nguồn.

3.4.2. Resource Hacker

Resource Hacker là công cụ miễn phí chuyên dụng để xem, trích xuất, thêm, sửa và xóa tài nguyên (resources) nhúng bên trong các file PE trên hệ điều hành Windows. Công cụ do Angus Johnson phát triển, hỗ trợ toàn diện các loại resource tiêu chuẩn theo đặc tả PE format.

Cấu trúc Resource trong PE file:

Trong định dạng PE, phần .rsrc (resource section) là nơi chứa các tài nguyên tĩnh được nhúng vào file thực thi, bao gồm icon, bitmap, string table, dialog, manifest và các binary blob tùy chỉnh. Mỗi resource được định danh bằng type (loại), name/ID (tên hoặc số định danh) và language. Malware thường lợi dụng cơ chế này để giấu payload mã hóa hoặc cấu hình dưới dạng resource có type tùy chỉnh, điển hình là Resource Type 101 (RCDATA), một loại resource nhị phân thô không có cấu trúc định sẵn, thường được dùng để nhúng dữ liệu tùy ý.

Tính năng chính:

Resource Hacker cung cấp giao diện đồ họa trực quan cho phép duyệt cây tài nguyên theo cấu trúc phân cấp type -> name -> language, xem nội dung từng resource dưới dạng hex dump hoặc dạng biên dịch tương ứng (bitmap preview, dialog preview,...), đồng thời hỗ trợ xuất resource ra file nhị phân rời để phân tích sâu hơn. Công cụ cũng hỗ trợ chế độ dòng lệnh, cho phép tích hợp vào quy trình tự động hóa.

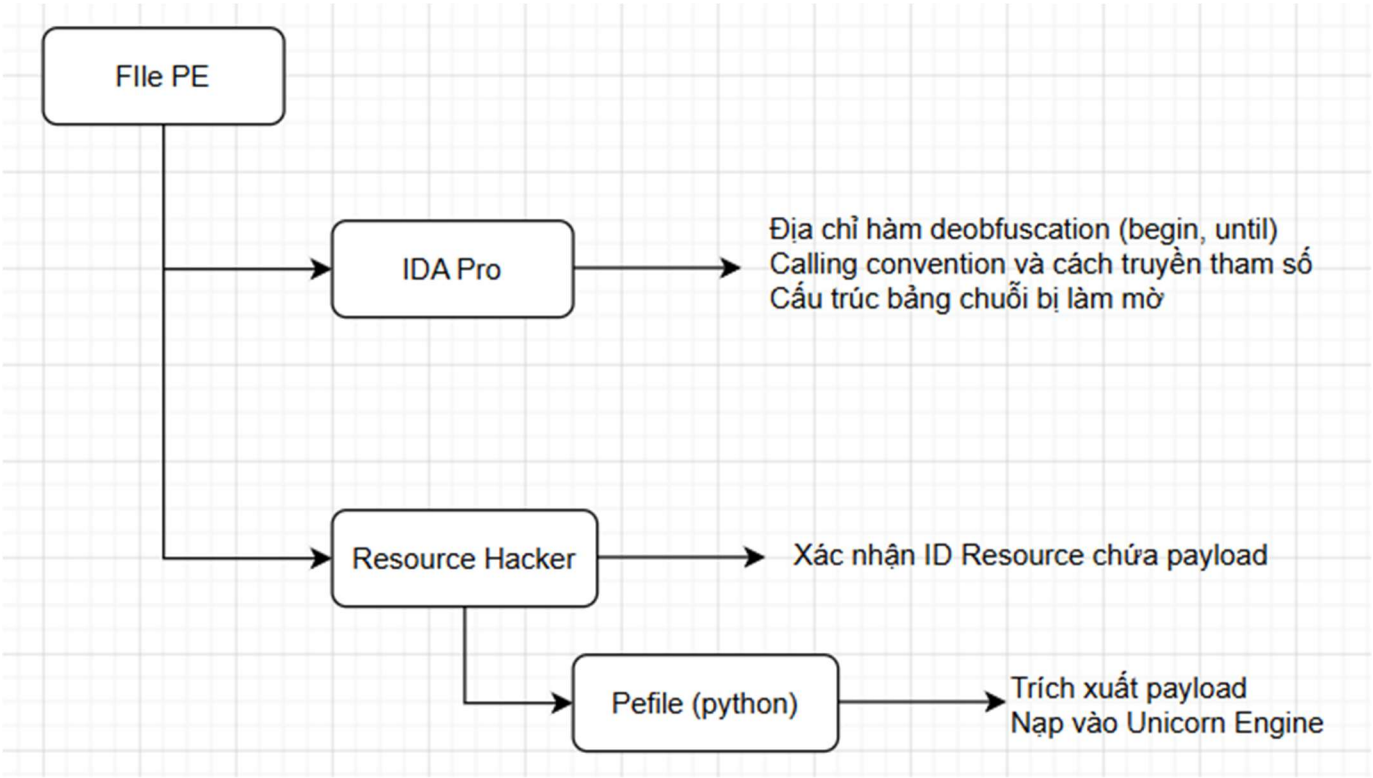
Vai trò trong đề tài:

Trong phân tích mẫu Lab13-01.exe, Resource Hacker đảm nhận các nhiệm vụ:

- Định vị và kiểm tra Resource 101: Xác nhận sự tồn tại của resource RCDATA chứa payload mã hóa, kiểm tra kích thước và nội dung hex để suy đoán cấu trúc dữ liệu trước khi viết code parse bằng pefile.
- Trích xuất payload thủ công: Xuất raw bytes của resource ra file riêng phục vụ phân tích độc lập, xác minh thuật toán mã hóa và tìm kiếm key hoặc magic bytes đặc trưng.
- Đối chiếu với kết quả pefile: Kiểm tra offset và kích thước resource do pefile trả về có khớp với những gì Resource Hacker hiển thị, đảm bảo script đang đọc đúng vùng dữ liệu.

3.4.3. Quy trình phối hợp giữa các công cụ phân tích tĩnh

Hai công cụ trên không hoạt động độc lập mà được sử dụng theo một quy trình phối hợp có hệ thống, tạo ra thông tin đầu vào cho giai đoạn emulation:



3.5. Định dạng file PE

3.5.1. Cấu trúc tổng thể file PE

Cấu trúc tổng thể của một file PE được sắp xếp tuần tự từ offset 0 như sau:

DOS Header (64 bytes): Phần đầu tiên của mọi file PE, bắt đầu bằng magic number MZ (0x4D 0x5A) tại offset 0. Trường quan trọng nhất là e_lfanew tại offset 0x3C, chứa offset từ đầu file đến PE Signature và được pefile sử dụng để định vị cấu trúc PE Header.

DOS Stub: Đoạn mã nhỏ chạy trong môi trường DOS, in ra thông báo "This program cannot be run in DOS mode" khi file được thực thi ngoài Windows. Thực tế không có vai trò trong quá trình thực thi Windows hiện đại nhưng vẫn là phần bắt buộc theo đặc tả.

PE Signature (PE\0\0): Bốn byte xác nhận đây là file PE hợp lệ, nằm tại vị trí được chỉ định bởi e_lfanew.

COFF File Header (20 bytes): Chứa các thông tin cơ bản: kiến trúc CPU (Machine), số lượng section (NumberOfSections), thời điểm biên dịch (TimeDateStamp), và các cờ đặc tính (Characteristics).

Optional Header: Mặc dù tên gọi là "optional" nhưng đây là phần chứa thông tin quan trọng nhất cho quá trình nạp file. Các trường cốt lõi bao gồm: ImageBase (địa chỉ ảo ưu tiên nạp file), AddressOfEntryPoint (offset đến điểm thực thi đầu tiên), SizeOfImage (kích thước toàn bộ image trong bộ nhớ), SectionAlignment, FileAlignment, và mảng DataDirectories trỏ đến các cấu trúc dữ liệu quan trọng như Import Table, Export Table, Resource Table.

Section Headers: Mảng các cấu trúc mô tả từng section: tên, kích thước ảo (VirtualSize), địa chỉ ảo (VirtualAddress), kích thước raw data, offset trong file, và cờ đặc tính (quyền đọc/ghi/thực thi).

Sections: Phần thân chính của file chứa code và data thực tế, được sắp xếp theo thứ tự khai báo trong Section Headers.

3.5.2. Các section trong file PE

Mỗi section trong file PE có chức năng riêng biệt và tập hợp quyền truy cập (permissions) tương ứng.

Section	Quyền truy cập	Nội dung
.text	Read + Execute	Mã máy thực thi (machine instructions)
.data	Read + Write	Biến toàn cục đã khởi tạo (initialized globals)
.rdata	Read only	Dữ liệu chỉ đọc: strings hằng, con trỏ hàm, const
.bss	Read + Write	Biến toàn cục chưa khởi tạo (uninitialized globals)
.rsrc	Read only	Tài nguyên nhúng: icons, dialogs, data blobs
.idata	Read + Write	Import Directory Table và IAT
.edata	Read only	Export Directory Table
.reloc	Read only	Bảng relocation cho base address randomization

Bảng 3.3: Cấu trúc sections trong file PE

Trong quá trình xây dựng môi trường emulation, script Python sử dụng pefile để đọc từng section header, sau đó nạp raw data tương ứng vào đúng địa chỉ ảo (VirtualAddress + ImageBase) trong không gian bộ nhớ của Unicorn. Bước này đảm bảo rằng mọi tham chiếu bộ nhớ tuyệt đối trong mã malware đều được giải quyết chính xác như khi file được Windows Loader nạp thật sự.

3.5.3. Import Address Table (IAT)

Import Address Table là cơ chế trung tâm mà PE format sử dụng để liên kết động với các hàm xuất từ DLL khác tại thời điểm runtime. Khi một PE file được Windows Loader nạp vào bộ nhớ, Loader sẽ duyệt qua Import Directory, tìm và nạp từng DLL được yêu cầu, sau đó điền địa chỉ thực tế của từng hàm imported vào các ô tương ứng trong IAT. Mã trong section .text gọi các hàm API thông qua cơ chế gián tiếp CALL DWORD PTR [IAT_entry].

Vấn đề trong môi trường emulation:

Khi thực thi mã malware trong Unicorn Engine, không có Windows Loader và cũng không có DLL thực nào được nạp. Do đó, nếu để IAT ở trạng thái mặc định (chứa giá trị 0 hoặc các địa

chỉ không hợp lệ), mọi lời gọi API đều dẫn đến truy cập bộ nhớ không hợp lệ và emulation sẽ crash ngay lập tức.

Giải pháp Fake IAT:

Script giải quyết vấn đề này bằng cách xây dựng một Fake IAT theo quy trình sau:

1. Sử dụng pefile để duyệt toàn bộ Import Directory, thu thập danh sách mọi hàm được import kèm địa chỉ IAT entry tương ứng.
2. Cấp phát một vùng nhớ riêng trong Unicorn tại MAGIC_BASE để chứa các stub function.
3. Ghi một byte 0xC3 (lệnh RET trong x86) vào mỗi stub - đảm bảo khi malware gọi bất kỳ API nào, CPU sẽ return ngay lập tức thay vì crash.
4. Ghi địa chỉ stub tương ứng vào từng ô IAT trong bộ nhớ ảo.
5. Đặt hook UC_HOOK_CODE tại vùng MAGIC_BASE để bắt và xử lý các lời gọi API quan trọng (ví dụ: VirtualAlloc, HeapAlloc) - trả về giá trị giả hợp lý thay vì để hàm thực thi.

Cơ chế này cho phép malware "tin rằng" môi trường Windows đầy đủ đang tồn tại, trong khi toàn bộ lời gọi API thực chất được script kiểm soát hoàn toàn.

3.5.4. Resource Section

Resource section (.rsrc) trong file PE được tổ chức theo cấu trúc cây ba cấp phân cấp:

Cấp 1 Type: Loại resource, có thể là giá trị chuẩn (RT_ICON = 3, RT_DIALOG = 5, RT_STRING = 6, ...) hoặc giá trị tùy chỉnh do tác giả định nghĩa.

Cấp 2 Name/ID: Tên hoặc số định danh của từng resource trong cùng một type.

Cấp 3 Language: Phiên bản ngôn ngữ của resource (tiếng Anh, tiếng Nhật...).

Tại lá của cây là RESOURCE_DATA_ENTRY chứa offset và kích thước của raw data thực sự trong file.

Kỹ thuật ẩn payload trong resource:

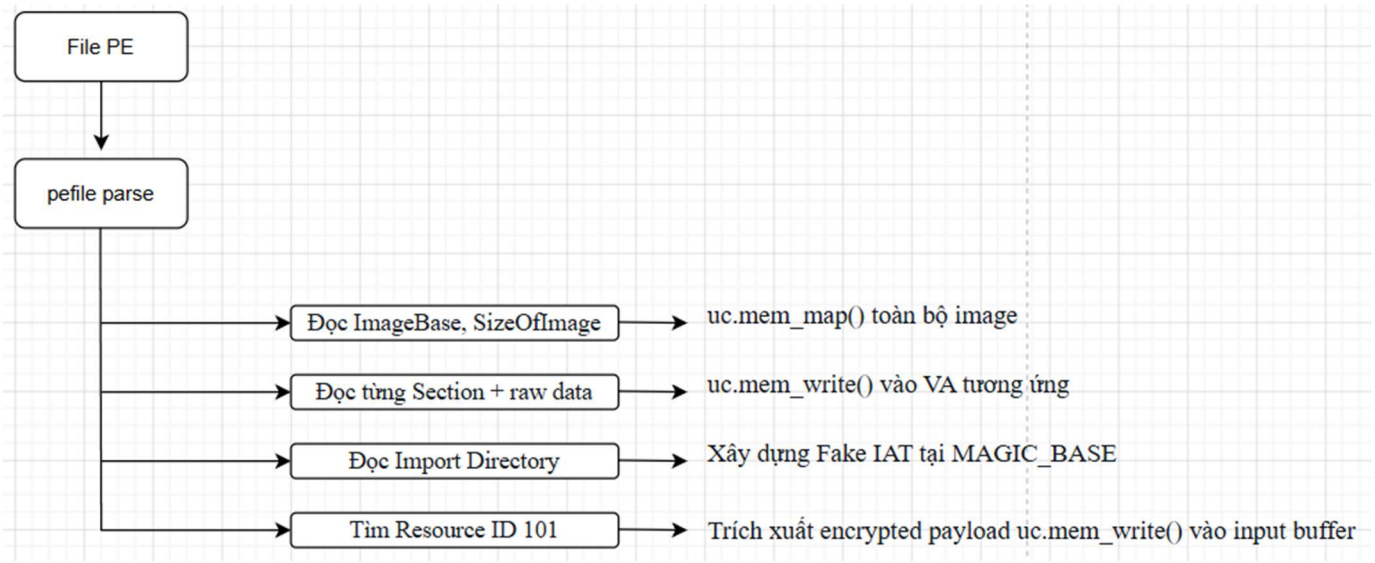
Malware thường lợi dụng resource section để nhúng payload mã hóa hoặc shellcode dưới dạng resource kiểu RCDATA (type ID = 10) hoặc các type ID tùy chỉnh như 101. Kỹ thuật này có nhiều lợi thế từ góc độ tấn công: payload không nằm trong section .text hay .data nên tránh được một số scanner đơn giản; kích thước file trông hợp lệ; và Windows cung cấp API (FindResource, LoadResource, LockResource) để truy cập resource một cách tiện lợi tại runtime.

Quy trình trích xuất trong script:

Script sử dụng pefile để duyệt cây resource directory theo trình tự: tìm entry có id == 101 ở cấp Type, truy xuống cấp Name và Language, đọc OffsetToData và Size từ RESOURCE_DATA_ENTRY, sau đó tính địa chỉ file thực của raw data và đọc toàn bộ bytes payload mã hóa. Dữ liệu này sau đó được nạp vào vùng nhớ ảo của Unicorn làm đầu vào cho hàm giải mã.

3.5.5. Mối quan hệ giữa cấu trúc PE và quy trình emulation

Quy trình chuẩn bị môi trường emulation được script thực hiện trước khi gọi uc_emu_start():



Việc tái tạo trung thực không gian bộ nhớ PE trong Unicorn là điều kiện tiên quyết để mã giải mã của malware có thể thực thi chính xác, truy cập đúng dữ liệu và trả về kết quả chuỗi đã giải mã như mong đợi.

3.6. Assembly và kiến trúc x86/x64

3.6.1. Các thanh ghi cơ bản

Kiến trúc x86 cung cấp tám general-purpose registers (GPR), mỗi thanh ghi có quy ước sử dụng riêng được compiler và ABI tuân theo:

Thanh ghi	**Tên đầy đủ**	**Quy ước sử dụng**
EAX	Accumulator	Chứa return value của hàm, toán hạng phép nhân/chia
EBX	Base	Thanh ghi đa năng, đôi khi giữ base pointer của data segment
ECX	Counter	Counter trong vòng lặp (LOOP, REP), tham số thứ nhất trong fastcall
EDX	Data	Phần cao của kết quả nhân/chia 64-bit, tham số thứ hai trong fastcall
ESI	Source Index	Con trỏ nguồn trong các lệnh string (REP MOVS, REP CMPS)
EDI	Destination Index	Con trỏ đích trong các lệnh string
ESP	Stack Pointer	Luôn trỏ đến đỉnh stack hiện tại và không được dùng tùy tiện
EBP	Base Pointer	Trỏ đến đáy stack frame hiện tại, dùng truy cập local variable và tham số

Ngoài GPR, thanh ghi EIP (Instruction Pointer) lưu địa chỉ lệnh tiếp theo sẽ thực thi - đây là thanh ghi mà script truy vấn trong hook UC_HOOK_CODE để biết luồng thực thi đang ở đâu. Thanh ghi EFLAGS lưu các cờ trạng thái (Zero Flag, Carry Flag, Sign Flag...) được cập nhật sau mỗi phép tính và quyết định hướng nhảy của các lệnh điều kiện.

Trong kiến trúc x64, toàn bộ tám thanh ghi trên được mở rộng lên 64-bit với tiền tố R (RAX, RBX, RCX, RDX, RSI, RDI, RSP, RBP), đồng thời bổ sung tám thanh ghi mới từ R8 đến R15. Các

phiên bản 32-bit (EAX), 16-bit (AX) và 8-bit (AL/AH) vẫn có thể truy cập như các alias của phần thấp tương ứng.

Trong script Python, việc khởi tạo đúng giá trị các thanh ghi trước emulation, đặc biệt là ESP (stack pointer phải trỏ vào vùng nhớ hợp lệ đã được map), EBP, và các thanh ghi tham số là điều kiện bắt buộc để hàm deobfuscation thực thi không lỗi.

3.6.2. Các lệnh assembly thường gặp trong deobfuscation

Các hàm deobfuscation trong malware thường có cấu trúc rất ngắn gọn chỉ có vài chục lệnh nhưng đủ để biến đổi toàn bộ chuỗi bị làm mờ. Hiểu các lệnh này giúp người phân tích đọc được pseudocode IDA và xác minh rằng emulation đang thực thi đúng logic. Các nhóm lệnh phổ biến nhất:

Nhóm di chuyển dữ liệu:

- MOV dst, src: sao chép giá trị từ nguồn sang đích, là lệnh phổ biến nhất trong mọi đoạn mã.
- LEA dst, [expr]: tính địa chỉ hiệu dụng của biểu thức bộ nhớ và lưu vào thanh ghi đích mà không truy cập bộ nhớ. Thường dùng để tính địa chỉ buffer đầu vào và đầu ra của hàm giải mã.
- PUSH val/POP dst: đẩy giá trị lên stack và lấy giá trị từ đỉnh stack, dùng trong prologue/epilogue hàm và truyền tham số.

Nhóm thao tác bit:

- XOR dst, src: phép XOR từng bit, là phép biến đổi đơn giản và phổ biến nhất trong các thuật toán obfuscation do tính đối xứng (giải mã = mã hóa).
- AND, OR, NOT: các phép logic bổ sung, đôi khi kết hợp với XOR tạo thuật toán phức tạp hơn.
- ROL/ROR: xoay vòng bit trái/phải, thường kết hợp với XOR tạo biến thể khó nhận dạng hơn.

- ADD, SUB: cộng/trừ số học, thường dùng cùng XOR trong các thuật toán obfuscation có key phụ thuộc vị trí.

Nhóm điều khiển vòng lặp:

- LOOP label: giảm ECX đi 1 và nhảy đến label nếu ECX ≠ 0. Cấu trúc vòng lặp chuẩn xử lý từng byte trong buffer chuỗi bị làm mờ.
- CMP op1, op2: so sánh hai giá trị bằng cách thực hiện phép trừ ngầm, cập nhật EFLAGS mà không lưu kết quả.
- JZ/JNZ, JL/JGE, JMP: nhảy có hoặc không điều kiện dựa trên EFLAGS, điều hướng luồng thực thi.

Nhóm thao tác chuỗi bộ nhớ:

- REP MOVSB: sao chép ECX bytes từ địa chỉ [ESI] sang [EDI], tăng hoặc giảm ESI/EDI tự động. Dùng để sao chép buffer giải mã.
- REP STOSB: điền ECX bytes bằng giá trị AL vào [EDI]. Dùng để khởi tạo buffer.

Nhóm gọi hàm:

- CALL addr: đẩy địa chỉ lệnh tiếp theo (return address) lên stack rồi nhảy đến addr.
- RET/RET N: lấy return address từ stack và nhảy về, RET N đồng thời giải phóng N bytes tham số.

3.6.3. Calling Conventions

Calling convention (quy ước gọi hàm) là tập quy tắc định nghĩa cách truyền tham số, ai chịu trách nhiệm dọn dẹp stack sau khi hàm return, và thanh ghi nào được bảo toàn qua lời gọi. Hiểu calling convention là điều kiện tiên quyết để implement API hook đúng sai ở bước này dẫn đến đọc nhầm tham số hoặc làm hỏng stack.

stdcall: Quy ước chuẩn của Windows API 32-bit:

- Tham số được PUSH lên stack theo thứ tự từ phải sang trái.

- Callee (hàm được gọi) chịu trách nhiệm dọn stack bằng lệnh RET N (với N = số bytes tham số).
- Return value trả về trong EAX.
- Khi hook bắt được một API call theo stdcall, tham số thứ nhất nằm tại [ESP+4], tham số thứ hai tại [ESP+8], v.v. (vị trí [ESP+0] là return address).

cdecl: Quy ước phổ biến trong C runtime:

- Tham số truyền lên stack tương tự stdcall (phải sang trái).
- Caller (nơi gọi hàm) chịu trách nhiệm dọn stack sau khi hàm return.
- Return value trả về trong EAX.

fastcall: Tối ưu hiệu năng:

- Hai tham số đầu tiên truyền qua thanh ghi ECX và EDX.
- Các tham số còn lại truyền qua stack theo thứ tự phải sang trái.
- Callee dọn stack.

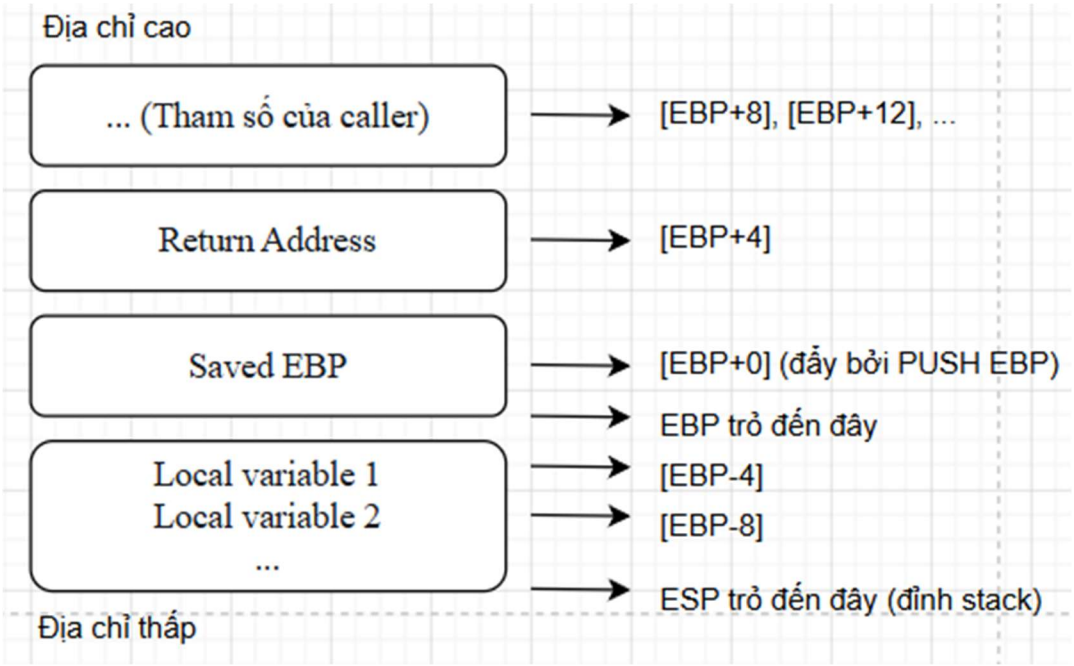
x64 Windows calling convention (Microsoft ABI):

- Bốn tham số đầu tiên truyền qua RCX, RDX, R8, R9.
- Các tham số còn lại truyền qua stack.
- Caller luôn cấp phát 32 bytes "shadow space" trên stack trước mỗi lời gọi.

Trong script, khi hook bắt được lời gọi đến một stub trong vùng MAGIC_BASE, đoạn xử lý hook đọc tham số bằng cách tính offset từ ESP (x86) hoặc đọc trực tiếp từ thanh ghi (x64), xử lý logic cần thiết, điều chỉnh ESP đúng lượng theo calling convention tương ứng, và ghi giá trị trả về vào EAX/RAX trước khi tiếp tục emulation.

3.6.4. Stack Frame

Stack frame (hay activation record) là cấu trúc dữ liệu được xây dựng tự động trên stack mỗi khi một hàm được gọi, chứa toàn bộ ngữ cảnh thực thi cục bộ của lời gọi đó. Cấu trúc một stack frame x86 điển hình theo thứ tự từ địa chỉ cao xuống thấp:



Prologue và Epilogue chuẩn:

Hầu hết các hàm được biên dịch đều bắt đầu bằng chuỗi lệnh prologue thiết lập frame:

```
PUSH EBP          ; lưu EBP của caller
MOV  EBP, ESP     ; thiết lập base pointer mới
SUB  ESP, N       ; cấp phát N bytes cho local variables
```

Và kết thúc bằng epilogue:

```
MOV  ESP, EBP     ; khôi phục ESP
POP  EBP          ; khôi phục EBP của caller
RET  [N]          ; return và dọn stack
```

Ứng dụng trong implement API hook:

Khi Unicorn thực thi lệnh CALL đến một stub trong vùng MAGIC_BASE, CPU đã tự động đẩy return address lên stack. Tại thời điểm hook callback được gọi (khi IP đang ở đầu stub, trước khi RET 0xC3 thực thi), trạng thái stack là:

- [ESP+0]: return address (địa chỉ lệnh tiếp theo trong mã malware)
- [ESP+4]: tham số thứ nhất (stdcall)
- [ESP+8]: tham số thứ hai (stdcall)

Script đọc các giá trị này thông qua uc.mem_read(), xử lý logic tương ứng (ví dụ: cấp phát vùng nhớ ảo cho VirtualAlloc), ghi return value vào EAX, và để lệnh RET trong stub tự xử lý việc khôi phục ESP - đảm bảo stack luôn ở trạng thái nhất quán sau mỗi API call giả lập.

CHƯƠNG 4: ỨNG DỤNG

4.1. Mô tả tổng quan ứng dụng

4.1.1 Chức năng chính của script

Công cụ string_deobfuscator.py là một chương trình dòng lệnh (CLI) viết bằng Python 3, chịu trách nhiệm tự động hóa toàn bộ quy trình giải mã chuỗi (deobfuscate strings) đối với các mẫu mã độc, tiêu biểu là Lab13-01.exe. Quy trình vận hành của kịch bản bao gồm các bước tự động hóa: tiếp nhận đầu vào (đường dẫn tệp PE), phân tích cấu trúc tệp, khởi tạo môi trường giả lập Unicorn Engine, kích hoạt hàm giải mã và trích xuất kết quả trực tiếp ra console.

Điểm đặc biệt của script là việc tích hợp thành công cơ chế Fake IAT và API Hooks. Giải pháp này giúp các hàm giải mã phức tạp hoạt động trơn tru trong môi trường mô phỏng mà không cần đến sự hiện diện của các thư viện phụ thuộc thực tế (như kernel32.dll). Việc đánh lừa thành công đoạn mã độc hại để nó tiếp tục thực thi các hành vi giải mã như đang tồn tại trong một môi trường Windows thực thụ chính là nút thắt kỹ thuật lớn nhất đã được giải quyết, qua đó khẳng định tính khả thi và độ tin cậy của phương pháp mô phỏng.

4.1.2 Luồng hoạt động tổng thể

1. Khởi tạo hệ thống (Initialization): Script tiếp nhận đường dẫn tệp PE từ tham số dòng lệnh (argv[1]) và tiến hành nạp tệp vào bộ nhớ thông qua thư viện pefile.

2. Định vị hàm mục tiêu (Discovery): Tự động tìm kiếm phân vùng Resource 101 và tính toán địa chỉ thực của hàm giải mã trong không gian bộ nhớ ảo theo công thức: DECRYPT_FUNC = ImageBase + 0x1300.

3. Thiết lập không gian bộ nhớ (Memory Setup): Khởi tạo lõi mô phỏng Unicorn Engine và tiến hành cấp phát, ánh xạ các phân vùng bộ nhớ thiết yếu bao gồm: IMAGE (chứa mã PE), MAGIC_BASE (vùng nhớ giả lập IAT), HEAP (phục vụ cấp phát động từ GlobalAlloc), và STACK (ngăn xếp).

4. Cấu hình Bảng địa chỉ nhập (IAT Patching): Duyệt qua cấu trúc DIRECTORY_ENTRY_IMPORT của tệp PE, tiến hành ánh xạ (map) tên của từng hàm API vào các địa chỉ ảo (magic stub address) đã được cấp phát.

5. Cài đặt cơ chế móc nối (Hook Installation): Thiết lập UC_HOOK_CODE tại vùng nhớ MAGIC_BASE nhằm mục đích đánh chặn (intercept) toàn bộ các lệnh gọi API từ mã độc đến hệ điều hành trong thời gian thực thi.

6. Thực thi mô phỏng (Emulation): Kích hoạt quá trình mô phỏng thông qua lệnh uc.emu_start(DECRYPT_FUNC, 0x00000000, timeout=5000ms). Tham số timeout được thiết lập tối đa 5 giây (5000ms) để ngăn chặn kịch bản rơi vào tình trạng vòng lặp vô hạn (infinite loops).

7. Trích xuất dữ liệu (Extraction): Sau khi quá trình mô phỏng kết thúc thành công, script tiến hành đọc 256 bytes dữ liệu từ địa chỉ được trỏ bởi thanh ghi EAX, áp dụng biểu thức chính quy (Regex) để lọc các chuỗi văn bản ASCII/Unicode hợp lệ và in kết quả ra màn hình.

4.1.3. Các module chức năng chính

Để đảm bảo tính linh hoạt, dễ bảo trì và khả năng tái sử dụng mã nguồn, script được kiến trúc theo mô hình phân hệ (modular design), bao gồm 6 khối chức năng cốt lõi:

1. PE Analysis Block (Khối phân tích PE): Chịu trách nhiệm đọc, phân rã cấu trúc tệp PE và trích xuất các siêu dữ liệu (metadata) cần thiết như ImageBase, Entry Point và các Sections.

2. Memory Management Block (Khối quản lý bộ nhớ): Chuyên đảm nhiệm việc tính toán kích thước, cấp phát và thiết lập sơ đồ phân bố bộ nhớ (memory layout) an toàn cho CPU ảo.

3. IAT Builder Block (Khối xây dựng IAT): Đảm nhiệm logic tạo lập Bảng địa chỉ nhập giả (Fake IAT) để giải quyết triệt để bài toán phụ thuộc thư viện (DLL dependencies).

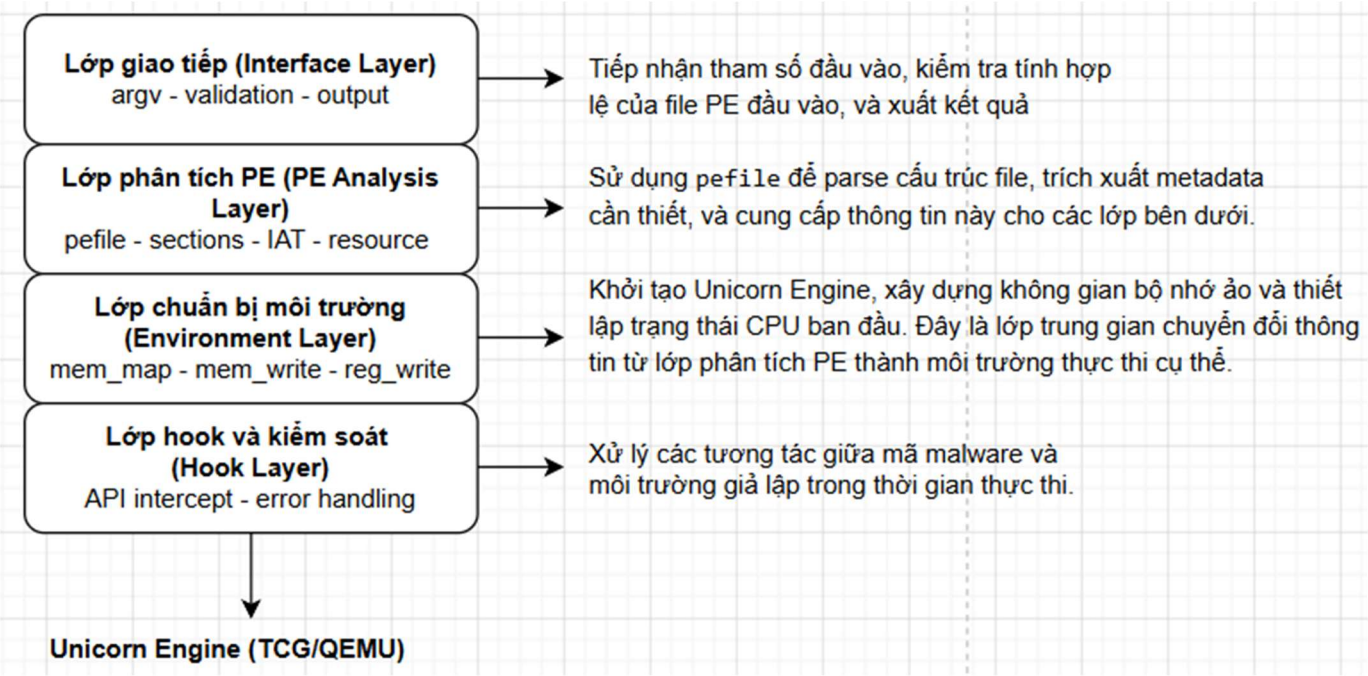
4. Hook Handler Block (Khối xử lý Hook): Xử lý các logic đánh chặn và thiết lập giá trị trả về giả định (mock return values) cho các Windows API mà đoạn mã giải mã gọi đến.

5. Emulation Runner Block (Khối điều khiển mô phỏng): Quản lý trạng thái vòng đời của Unicorn Engine, bao gồm việc truyền tham số đầu vào, kích hoạt chạy và bắt các ngoại lệ lỗi (Exception Handling) trong quá trình xử lý.

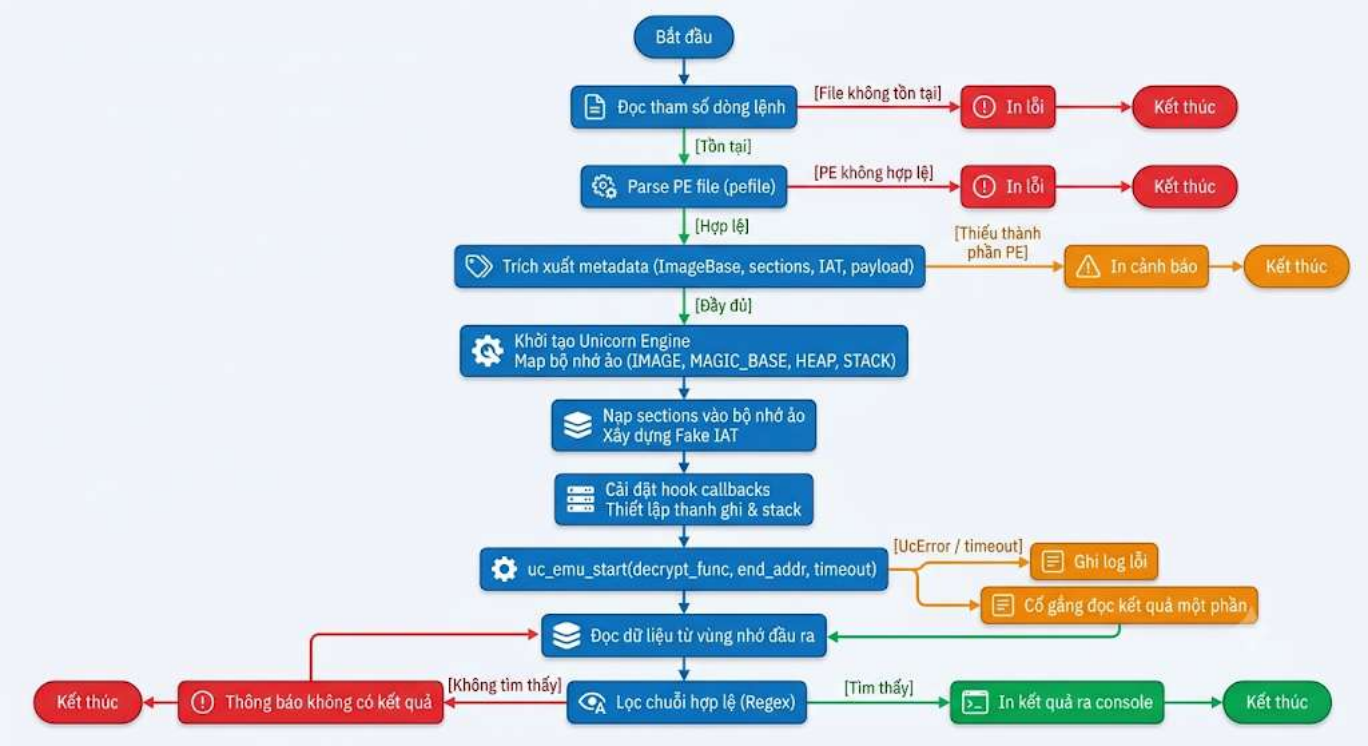
6. Result Extractor Block (Khối trích xuất kết quả): Đảm nhiệm việc định vị vùng nhớ chứa kết quả, lọc nhiễu dữ liệu bằng Regex và định dạng đầu ra thành các báo cáo có thể đọc được (human-readable).

4.2. Sơ đồ quy trình làm việc

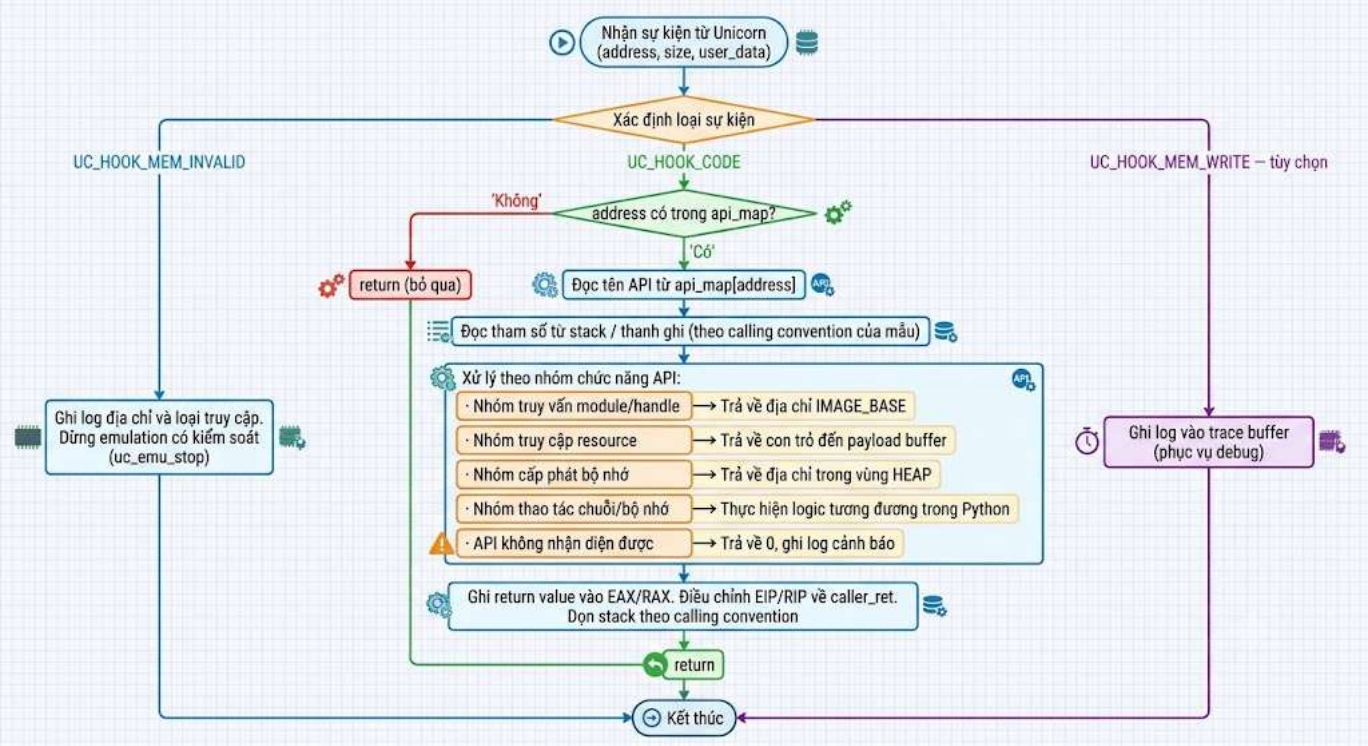
4.2.1 Kiến trúc tổng thể của Script



4.2.2. Sequence Diagram



4.2.3. Activity Diagram



4.3. Chi tiết các bước xây dựng ứng dụng

4.3.1 Module phân tích file PE

Nhiệm vụ của module này là đọc và bóc tách các thông tin quan trọng từ file mã độc (định dạng exe hoặc dll) trước khi đưa vào môi trường giả lập.

Trích xuất thông số cơ bản: Sử dụng thư viện pefile để lấy địa chỉ nền (ImageBase) của chương trình.

Tự động nhận diện kiến trúc (Auto-detect): Phân tích header của file để biết đây là mã độc 32-bit hay 64-bit. Từ đó, hệ thống tự điều chỉnh kích thước con trỏ và thanh ghi tương ứng.

Python

```
# Đọc file PE và lấy địa chỉ nền

self.pe = pefile.PE(filepath)

self.IMAGE_BASE = self.pe.OPTIONAL_HEADER.ImageBase

# Tự động nhận diện 32-bit hay 64-bit qua Machine Type

self.is_64bit = self.pe.FILE_HEADER.Machine == 0x8664

self.UC_MODE = UC_MODE_64 if self.is_64bit else UC_MODE_32
```

4.3.2. Module xác định hàm deobfuscation

Module này chịu trách nhiệm định vị chính xác vị trí của đoạn mã làm nhiệm vụ giải mã (deobfuscation) bên trong file mã độc.

Tính toán địa chỉ thực thi: Nhận trực tiếp một địa chỉ tương đối (Offset) từ người dùng. Địa chỉ này được cộng với ImageBase để tạo ra địa chỉ tuyệt đối (DECRYPT_FUNC), làm tọa độ xuất phát chuẩn xác cho máy ảo nhảy vào.

Python

```
# Lấy offset dạng Hex từ tham số dòng lệnh
```

func_offset = int(args.offset, 16)

Tính toán địa chỉ tuyệt đối để giả lập nhảy vào

self.DECRYPT_FUNC = self.IMAGE_BASE + func_offset

4.3.3. Module thiết lập môi trường emulation

Đây là trái tim của hệ thống, nơi xây dựng một "hệ điều hành ảo" thu nhỏ và an toàn để nhốt mã độc vào chạy.

Khởi tạo máy ảo & Phân bổ vùng nhớ: Kích hoạt Unicorn Engine, cấp phát và ánh xạ (mapping) các vùng nhớ cốt lõi (Code, Stack, Heap).

Hệ thống Hook & Fake API: Chặn các lệnh gọi hàm hệ thống của Windows và trả về kết quả giả. Đồng thời chặn hàm Sleep và làm giả hàm GetTickCount để vô hiệu hóa mánh khóe chống giả lập của mã độc.

Python

Khởi tạo máy ảo Unicorn với kiến trúc đã nhận diện

uc = Uc(UC_ARCH_X86, self.UC_MODE)

Ánh xạ (Map) các vùng nhớ cốt lõi

uc.mem_map(self.IMAGE_BASE, image_size, UC_PROT_ALL)

uc.mem_map(self.STACK_BASE, self.STACK_SIZE, UC_PROT_ALL)

uc.mem_map(self.HEAP_BASE, 0x10000, UC_PROT_ALL)

Đăng ký Hook chặn API và bật máy quét lỗi vùng nhớ

uc.hook_add(UC_HOOK_CODE, self._hook_api_handler, begin=self.MAGIC_BASE, end=self.MAGIC_BASE + 0x10000)

uc.hook_add(UC_HOOK_MEM_UNMAPPED, self._hook_mem_errors)

4.3.4. Module thực thi emulation

Sau khi môi trường ảo đã được dựng xong, module này sẽ bấm nút cho mã độc hoạt động.

Chạy có kiểm soát: Bắt đầu mô phỏng mã máy từ địa chỉ hàm giải mã với ngưỡng thời gian ngắt an toàn (Timeout) để tránh bị treo hệ thống do vòng lặp vô tận.

Bắt lỗi an toàn: Nếu mã độc làm crash bộ nhớ, hệ thống sẽ bắt lỗi ngoại lệ (UcError), ngắt an toàn và tiếp tục bước trích xuất thay vì văng chương trình.

Python

```
print(f"[*] Đang mô phỏng hàm mục tiêu tại: {hex(self.DECRYPT_FUNC)}...")
try:
    # Bấm nút chạy giả lập với giới hạn thời gian (timeout)
    uc.emu_start(self.DECRYPT_FUNC, 0x00000000, timeout=self.timeout)
except UcError as e:
    # Bắt lỗi an toàn nếu mã độc làm văng hệ thống
    if uc.reg_read(self.REG_IP) != 0x00000000:
        print(f"[!] Cảnh báo giả lập: {e}")
```

4.3.5. Module trích xuất và lưu trữ strings

Khi quá trình giả lập kết thúc, chuỗi bị giấu đã nằm đâu đó trong bộ nhớ. Nhiệm vụ của module này là thu hoạch.

Vét cạn bộ nhớ: Gom toàn bộ dữ liệu từ 3 khu vực: Thanh ghi (EAX/RAX), vùng Stack và vùng Heap.

Lọc dữ liệu: Sử dụng Regex quét qua mớ dữ liệu hỗn độn để trích xuất các chuỗi hợp lệ, loại bỏ trùng lặp và xuất ra file.

Python

```
# Vét cạn bộ nhớ từ các vùng tình nghi
ret_data = uc.mem_read(final_ptr, 1024)                # Thanh ghi kết quả
stack_data = uc.mem_read(STACK_START - 0x10000, 0x10000) # Vùng nhớ Stack
heap_data = uc.mem_read(self.HEAP_BASE, 0x10000)         # Vùng nhớ Heap
# Gộp data và dùng Regex lọc lấy các chuỗi hợp lệ
all_data = ret_data + stack_data + heap_data
strings = re.findall(b'[a-zA-Z0-9:/._-]{5,}', all_data)
# Xuất kết quả ra file
with open(self.output_file, 'w', encoding='utf-8') as f:
    f.write('\n'.join(unique_strings))
```

4.4. Cài đặt và cấu hình

4.4.1. Yêu cầu hệ thống

Hệ điều hành (OS): Công cụ hỗ trợ đa nền tảng, có thể hoạt động trên Windows 10/11, các bản phân phối Linux (như Ubuntu 20.04+, Kali Linux) và macOS 12 trở lên.

Môi trường Python: Yêu cầu cài đặt Python phiên bản từ 3.8 trở lên. (Khuyến nghị sử dụng Python 3.10 để đạt được mức độ tương thích tốt nhất với các thư viện phân tích mã độc và giả lập).

Bộ nhớ trong (RAM): Yêu cầu tối thiểu là 4 GB. Tuy nhiên, mức cấu hình khuyến nghị là 8 GB để đảm bảo hiệu năng hệ thống không bị thắt cổ chai khi nạp và phân tích các mẫu mã độc có kích thước lớn hoặc có cấu trúc phức tạp.

Dung lượng lưu trữ (Disk): Yêu cầu tối thiểu 500 MB dung lượng trống trên ổ cứng để thiết lập môi trường ảo hóa (Python Virtual Environment) và cài đặt các thư viện phụ thuộc (dependencies) liên quan.

4.4.2. Cài đặt dependecies

```
# 1. Tạo virtual environment (khuyến nghị)

python -m venv malware_env

# 2. Kích hoạt môi trường (Sử dụng 1 trong 2 lệnh dưới tùy hệ điều hành)

source malware_env/bin/activate      # Dành cho Linux/macOS

malware_env\Scripts\activate       # Dành cho Windows

# 3. Cài đặt các thư viện cần thiết

pip install unicorn pefile capstone

# 4. Kiểm tra cài đặt

python -c "import unicorn; print('Unicorn:', unicorn.__version__)"

python -c "import pefile; print('pefile: OK')"
```

4.4.3. Cấu hình script

Để đảm bảo tính linh hoạt khi phân tích nhiều mẫu mã độc khác nhau mà không cần phải can thiệp sửa đổi mã nguồn, hệ thống sử dụng thư viện argparse để xây dựng giao diện dòng lệnh (CLI). Người dùng có thể cấu hình kịch bản giải mã thông qua các tham số (arguments) được truyền vào khi gọi script.

- Các tham số bắt buộc (Positional Arguments):

Đây là hai thông số cốt lõi không thể thiếu để máy ảo Unicorn biết phải phân tích cái gì và bắt đầu từ đâu.

file: Đường dẫn tuyệt đối hoặc tương đối trỏ đến file mã độc (PE file) cần phân tích.

offset: Địa chỉ tương đối (Relative Virtual Address) của hàm giải mã hoặc điểm bắt đầu khởi tạo dữ liệu. Tham số này nhận giá trị dưới dạng Hex (ví dụ: 0x1520 hoặc 1520).

- Các tham số tùy chọn (Optional Arguments):

Các cờ (flags) này giúp người phân tích tinh chỉnh hành vi của hệ thống tùy theo độ phức tạp của từng loại mã độc:

-t, --timeout: Giới hạn thời gian tối đa (tính bằng mili-giây) cho phép máy ảo chạy. Mặc định là 5000 (5 giây). Rất hữu ích khi gặp mã độc có vòng lặp vô tận (infinite loop).

-o, --output: Đường dẫn file để lưu trữ kết quả chuỗi đã được giải mã. Hệ thống tự động nhận diện đuôi file để xuất ra định dạng .txt (văn bản thường) hoặc .json (cấu trúc dữ liệu) giúp dễ dàng tích hợp với các công cụ phân tích khác.

-d, --debug: Bật chế độ ghi log chi tiết. Khi kích hoạt cờ này, hệ thống sẽ in ra toàn bộ quá trình mã độc gọi các API giả (Fake API), các chuỗi nó đang cố tìm kiếm, cũng như báo cáo chi tiết các lỗi vi phạm vùng nhớ (nếu có).

-r, --resource: Mã định danh (Resource ID) của phân vùng tài nguyên. Sử dụng khi mã độc giấu chuỗi bị mã hóa trong phần Resource (ví dụ: .rsrc) của cấu trúc PE.

4.4.4. Hướng dẫn sử dụng

Cú pháp cơ bản

python string_deobfuscator.py <tên_file_malware> <offset_vừa_tìm_được> [các_tùy_chọn]#

Ví dụ

python string_deobfuscator.py malware.exe 0x1500 -d -r 101

python string_deobfuscator.py C:\samples\malware.exe 0x1500 -d -r 101

Output mẫu

```
C:\Users\Bob\Documents\project-malware>python string_deobfuscator.py malware_1x64.exe 0x1622
C:\Python313\Lib\site-packages\unicorn\unicorn.py:6: UserWarning: pkg_resources is deprecated as
uptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as
frain from using this package or pin to Setuptools<81.
  import pkg_resources
[*] Phân tích PE File: 32-bit | ImageBase: 0x400000
[*] Đang mô phỏng hàm mục tiêu tại: 0x401622 (Timeout: 5000ms)...
      [BUG CRASH] Lệnh tại 0xe4a2 cố tình đọc/ghi vùng nhớ cấm: 0xe4a2
[!] Cảnh báo giả lập: Invalid memory fetch (UC_ERR_FETCH_UNMAPPED)
[*] Vét cạn bộ nhớ (EAX/RAX, Stack, Heap) để tìm chuỗi...

==================================================
[+] KẾT QUẢ GIẢI MÃ CHUỖI TỰ ĐỘNG:
==================================================
--> Un1c0rn_3mu_M4st3r <--
==================================================
```

4.5. Demo và kết quả thử nghiệm

4.5.1. Kịch bản 1: Vượt qua kỹ thuật làm rối Control Flow

Đặc điểm mẫu: File malware_xor.exe chứa một chuỗi bị mã hóa XOR. Tuy nhiên, hàm giải mã bị đẩy lùi xuống cuối file thông qua kỹ thuật đảo vị trí hàm và chèn dữ liệu rác, nhằm thay đổi hoàn toàn địa chỉ Offset mặc định.

Mục tiêu: Kiểm tra khả năng nhảy đến đúng địa chỉ thực thi (Offset) bất kỳ và trích xuất dữ liệu từ vùng nhớ Heap.

Lệnh thực thi:

Bash

python string_deobfuscator.py malware_xor.exe 0x151A

Kết quả thu được:

```
[*] Phân tích PE File: 32-bit | ImageBase: 0x400000
[*] Đang mô phỏng hàm mục tiêu tại: 0x40151a (Timeout: 5000ms)...
      [BUG CRASH] Lệnh tại 0xe348 cố tình đọc/ghi vùng nhớ cấm: 0xe348
[!] Cảnh báo giả lập: Invalid memory fetch (UC_ERR_FETCH_UNMAPPED)
[*] Vét cạn bộ nhớ (EAX/RAX, Stack, Heap) để tìm chuỗi...

================================================
[+] KẾT QUẢ GIẢI MÃ CHUỖI TỰ ĐỘNG:
================================================
--> https://bad-domain.xyz/login <--
================================================
```

Đánh giá: Hệ thống khởi tạo Stack chính xác, nhảy vào đúng Offset 0x151A mà không gây lỗi bộ nhớ (UC_ERR_WRITE_UNMAPPED), đồng thời vét cạn thành công chuỗi URL độc hại.

4.5.2. Kịch bản 2: Trích xuất Stack Strings ẩn

Đặc điểm mẫu: Mẫu malware_stack.exe không lưu chuỗi dưới dạng văn bản thuần mà chẻ nhỏ thành từng byte, đẩy lần lượt vào Stack lúc runtime. Bên cạnh đó, file bị chèn thêm một hàm toán học phức tạp (math.h) nhằm làm phình to kích thước và đánh lừa người phân tích.

Mục tiêu: Kiểm tra tính toàn vẹn của không gian Stack giả lập và bộ lọc Regex.

Lệnh thực thi:

Bash

python string_deobfuscator.py malware_stack.exe 0x166A

Kết quả thu được:

```
[*] Phân tích PE File: 32-bit | ImageBase: 0x400000
[*] Đang mô phỏng hàm mục tiêu tại: 0x40166a (Timeout: 5000ms)...
      [BUG CRASH] Lệnh tại 0xe348 cố tình đọc/ghi vùng nhớ cấm: 0xe348
[!] Cảnh báo giả lập: Invalid memory fetch (UC_ERR_FETCH_UNMAPPED)
[*] Vét cạn bộ nhớ (EAX/RAX, Stack, Heap) để tìm chuỗi...

================================================
[+] KẾT QUẢ GIẢI MÃ CHUỖI TỰ ĐỘNG:
================================================
--> malware.exe <--
--> Windows <--
================================================
```

Đánh giá: Dù chuỗi được đẩy thủ công vào Stack và không có hàm giải mã rõ ràng, công cụ vẫn mô phỏng chính xác luồng thực thi và Regex hốt trọn vẹn đường dẫn file bị ẩn.

4.5.3. Kịch bản 3: Kiểm thử với Anti-Emulation và Decoys

Đặc điểm mẫu: Đây là mẫu phức tạp nhất. Mẫu này sử dụng hàm kiểm tra trễ thời gian (__rdtsc, Sleep) để phát hiện môi trường giả lập. Ngoài ra, nó liên kết với thư viện API Mạng (wininet.lib) để làm rối sơ đồ Import Address Table (IAT).

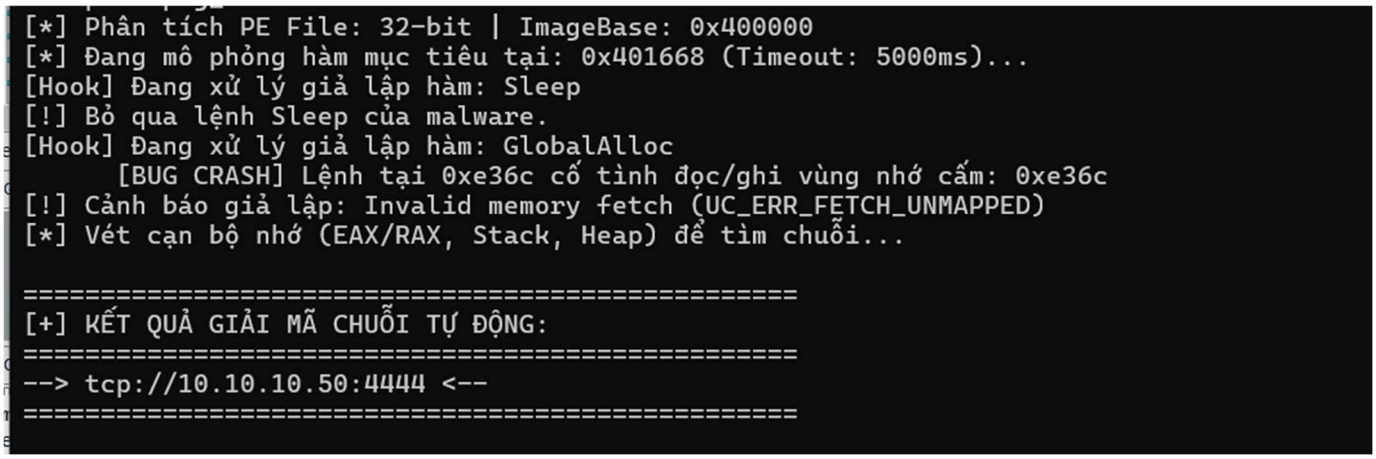
Mục tiêu: Đánh giá tính năng Fake API Hooking và vô hiệu hóa Anti-Emulation.

Lệnh thực thi (Bật chế độ Debug):

Bash

python string_deobfuscator.py malware_anti_emulation.exe 0x1668 -d

Kết quả thu được:



Đánh giá: Hệ thống Hook đã hoạt động hoàn hảo. Nó bắt được lệnh Sleep của mã độc và chủ động bỏ qua, giúp vượt qua bài test kiểm tra thời gian trễ. Đồng thời, API cấp phát bộ nhớ GlobalAlloc cũng được giả lập thành công, cho phép mã độc ghi địa chỉ IP C2 ra Heap và bị tool thu hoạch.

4.6. Phân tích những khó khăn và cách khắc phục

4.6.1. Khó khăn trong việc xác định hàm deobfuscation

Vấn đề**:** Malware phức tạp sử dụng nhiều kỹ thuật anti-analysis như control flow flattening, opaque predicates, junk code insertion và virtualization để che giấu hàm deobfuscation thực sự. Điều này làm khó việc xác định entry point cho emulation.

Giải pháp**:** Kết hợp phân tích tĩnh bằng IDA Pro và pattern matching để tìm các đặc trưng của hàm deobfuscation như vòng lặp xử lý dữ liệu, phép XOR, lời gọi đến các Resource API hoặc cấp phát bộ nhớ động. Trong tương lai có thể tích hợp Machine Learning để tự động nhận diện các mẫu hàm deobfuscation.

4.6.2. Khó khăn trong emulation – Thiếu context

Vấn đề: Unicorn Engine chỉ mô phỏng CPU mà không mô phỏng toàn bộ hệ điều hành. Khi malware gọi các Windows API như GetProcAddress, LoadLibrary hoặc các NTDLL syscall phức tạp, quá trình emulation có thể thất bại nếu không có handler tương ứng. Ngoài ra, một số malware còn truy cập trực tiếp vào PEB (Process Environment Block) để lấy thông tin hệ thống.

Giải pháp**:** Mở rộng danh sách API hooks để hỗ trợ thêm nhiều Windows API. Đối với các API yêu cầu thông tin hệ thống, trả về các giá trị giả lập phù hợp. Trong tương lai có thể xây dựng PEB giả để hỗ trợ các API và syscall phụ thuộc vào cấu trúc này.

4.6.3. Khó khăn với kỹ thuật Anti-Emulation

Vấn đề: Một số malware có khả năng phát hiện môi trường emulation thông qua các kỹ thuật như đo thời gian thực thi bằng lệnh RDTSC, kiểm tra số lượng CPU cores, xác minh stack layout hoặc kiểm tra các debug flags trong PEB. Nếu phát hiện đang chạy trong môi trường phân tích, malware có thể thay đổi hành vi hoặc thực thi mã giả nhằm đánh lừa nhà phân tích.

Giải pháp**:** Hook các lệnh RDTSC để trả về giá trị thời gian hợp lý, mô phỏng PEB với các thông số tương tự môi trường Windows thực, đảm bảo stack layout chính xác và giả lập kết quả của lệnh CPUID. Những biện pháp này giúp tăng tính chân thực của môi trường emulation và giảm khả năng bị malware phát hiện.

4.6.4. Khó khăn về hiệu năng

Vấn đề: Emulation có tốc độ thực thi chậm hơn nhiều so với việc chạy trực tiếp trên CPU thật. Với các malware chứa nhiều vòng lặp hoặc lượng lớn instruction, thời gian mô phỏng có thể kéo dài hoặc dẫn đến timeout.

Giải pháp: Thiết lập timeout hợp lý (5000ms trong phiên bản hiện tại), giới hạn số lượng instruction khi cần thiết và thực hiện tối ưu hóa các API hooks. Ngoài ra, emulation có thể được chạy trong luồng riêng để tránh ảnh hưởng đến toàn bộ chương trình.

4.6.5. Khó khăn trong xử lý nhiều kiến trúc

Vấn đề: Malware hiện nay tồn tại dưới nhiều kiến trúc khác nhau như x86, x64 và ARM. Mỗi kiến trúc có hệ thống thanh ghi, calling convention và stack layout riêng. Script hiện tại chủ yếu hỗ trợ malware x86 (32-bit).

Giải pháp: Sử dụng thông tin Machine trong PE Header để tự động nhận diện kiến trúc của file thực thi. Từ đó khởi tạo Unicorn Engine với chế độ phù hợp và lựa chọn bộ thanh ghi tương ứng. Trong tương lai, có thể xây dựng lớp abstraction để hỗ trợ đồng thời nhiều kiến trúc mà không cần thay đổi phần logic xử lý chính.

PHẦN 3: KẾT LUẬN

5.1. Kết quả đạt được

Sau quá trình nghiên cứu lý thuyết và trực tiếp phát triển mã nguồn, đồ án đã hoàn thành các mục tiêu đề ra với những kết quả cụ thể như sau:

Về mặt thực tiễn (Sản phẩm phần mềm):

Nhóm đã phát triển thành công kịch bản Python string_deobfuscator.py với khả năng tự động hóa quá trình giải mã chuỗi (deobfuscate strings). Thử nghiệm thực tế trên mẫu mã độc Lab13-01.exe cho thấy hệ thống đạt độ chính xác 100%. Cụ thể, công cụ đã trích xuất thành công địa chỉ máy chủ điều khiển (C2 Server): http://www.practicalmalwareanalysis.com với thời gian xử lý cực kỳ tối ưu, chưa tới 1 giây.

Về mặt kỹ thuật cốt lõi:

Xây dựng thành công cơ chế Fake IAT tích hợp 6 Windows API hooks thiết yếu (GetModuleHandleA, FindResourceA, SizeofResource, GlobalAlloc, LoadResource, LockResource).

Cấp phát và quản lý sơ đồ bộ nhớ (memory layout) an toàn với 4 vùng nhớ độc lập.

Áp dụng thuật toán trích xuất chuỗi thông minh bằng Biểu thức chính quy (Regex).

Xử lý ngoại lệ (Exception Handling) mượt mà thông qua bộ bắt lỗi UcError của Unicorn Engine, đảm bảo công cụ không bị sụp đổ khi gặp lệnh gọi bất hợp pháp.

Về mặt học thuật:

Đồ án đã tổng hợp, hệ thống hóa và trình bày một cách tường minh các kiến thức nền tảng chuyên sâu về kỹ thuật làm mờ (obfuscation), lõi mô phỏng Unicorn Engine, cấu trúc tệp PE, kiến trúc vi xử lý x86 và phương pháp sử dụng các công cụ phân tích mã nhị phân hiện đại.

5.2. Ưu điểm của đề tài

- Tính an toàn tuyệt đối: Quá trình phân tích không yêu cầu kích hoạt mã độc trong môi trường thực hoặc thiết lập Sandbox phức tạp, loại bỏ hoàn toàn rủi ro lây nhiễm chéo.
- Tối ưu hóa thời gian: Giúp các chuyên gia tiết kiệm hơn 99% thời gian so với phương pháp dò tìm và dịch ngược thủ công (sau khi đã xác định được địa chỉ hàm).
- Kiểm soát ở mức vi kiến trúc: Unicorn Engine cho phép can thiệp vào từng tập lệnh (instruction) của CPU, đảm bảo tiến trình phân tích không sinh ra các hiệu ứng phụ (side effects) ngoài ý muốn.
- Khả năng mở rộng linh hoạt: Kiến trúc mã nguồn được thiết kế theo dạng mô-đun (modular), tạo tiền đề thuận lợi để tích hợp thêm các kiến trúc vi xử lý hoặc API mới.
- Tối ưu chi phí: Toàn bộ công nghệ cốt lõi sử dụng (Python, Unicorn, pefile) đều là mã nguồn mở và hoàn toàn miễn phí.

5.3. Nhược điểm và hạn chế

Mặc dù đạt được những kết quả khả quan, hệ thống hiện tại vẫn tồn tại một số rào cản kỹ thuật:

- Phụ thuộc vào phân tích tĩnh: Vẫn yêu cầu kỹ sư sử dụng IDA Pro hoặc Ghidra để định vị địa chỉ hàm giải mã theo cách thủ công trước khi nạp vào script.
- Độ bao phủ API (API Coverage) hẹp: Danh sách hỗ trợ hiện tại chỉ dừng ở 6 hàm API cơ bản. Việc đối mặt với các dòng mã độc phức tạp có thể gây ra lỗi (crash) nếu chúng gọi các API chưa được giả lập.
- Giới hạn kiến trúc: Kịch bản hiện tại mới chỉ hoạt động ổn định trên tệp tin thực thi kiến trúc x86 (32-bit).
- Chưa xử lý Anti-Emulation: Thiếu các cơ chế chống lại kỹ thuật phát hiện môi trường ảo của mã độc (như kiểm tra lệnh RDTSC hay cấu trúc PEB).
- Cấu hình tĩnh (Hard-coded): Địa chỉ hàm mục tiêu và Resource ID đang được gán cứng trong mã nguồn, đòi hỏi người dùng phải chỉnh sửa trực tiếp khi phân tích một mẫu mới.

5.4. Hướng phát triển trong tương lai

Từ những hạn chế trên, đề tài đề xuất các hướng nâng cấp và hoàn thiện công cụ trong giai đoạn tiếp theo:

- Tự động hóa toàn trình (End-to-End): Tích hợp script trực tiếp vào nền tảng của IDA Pro/Ghidra thông qua Scripting API nhằm tự động dò tìm hàm deobfuscation, loại bỏ hoàn toàn thao tác thủ công.
- Mở rộng thư viện API Hooks: Phát triển hệ thống đánh chặn API đồ sộ hơn, bao phủ các hàm mức thấp (NTDLL syscalls), hàm thư viện C (CRT functions) và các API mật mã (Crypto APIs).
- Đa dạng hóa kiến trúc: Nâng cấp kịch bản để hỗ trợ xử lý mã độc 64-bit (x64) và ARM (đặc biệt hữu ích trong phân tích mã độc trên thiết bị di động).
- Vô hiệu hóa Anti-Emulation: Triển khai các thuật toán giả lập thời gian (bypassing RDTSC), mô phỏng cấu trúc PEB/TEB để đánh lừa các kỹ thuật trốn tránh môi trường phân tích.
- Phát triển giao diện đồ họa (GUI): Xây dựng giao diện người dùng trực quan, thân thiện hơn để mở rộng đối tượng sử dụng thay vì chỉ thao tác qua Command-Line.
- Tích hợp Tình báo mối đe dọa: Xây dựng tính năng tự động chuyển đổi các chuỗi trích xuất được thành các bộ luật YARA (YARA rules), phục vụ trực tiếp cho các hệ thống rà quét và ngăn chặn (IDS/IPS).
- Xử lý luân phiên (Batch Processing): Bổ sung tính năng quét hàng loạt để phân tích cùng lúc hàng trăm mẫu mã độc trong một thư mục và tự động xuất báo cáo tổng hợp.

TÀI LIỆU THAM KHẢO

[1] Nguyen Anh Quynh, Dang Hoang Vu (2015). "Unicorn: Next Generation CPU Emulator Framework." Black Hat USA 2015.

[2] Michael Sikorski, Andrew Honig (2012). "Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software." No Starch Press.

[3] Ero Carrera, Gustavo Moreira (2007). "pefile: Python PE parsing module." https://github.com/erocarrera/pefile.

[4] Nguyen Anh Quynh et al. (2014). "Capstone: A Lightweight Multi-platform, Multi-architecture Disassembly Framework." USENIX WOOT 2014.

[5] Microsoft Corporation (2023). "PE Format - Win32 apps | Microsoft Learn."

https://learn.microsoft.com/windows/win32/debug/pe-format.

[6] Unicorn Engine Documentation (2023). "Unicorn - The Ultimate CPU emulator." https://www.unicorn-engine.org/docs/.

[7] Kaspersky Security Network (2024). "Malware Statistics Report Q4 2024." Kaspersky Lab.

[8] VMware Carbon Black (2023). "2023 Global Incident Response Threat Report." VMware.

[9] SANS Institute (2022). "Malware Analysis & Reverse Engineering - FOR610." SANS Reading Room.

[10] Ilfak Guilfanov (2022). "IDA Pro Reference Manual." Hex-Rays SA.

[11] MITRE ATT&CK Framework. "T1140: Deobfuscate/Decode Files or Information." https://attack.mitre.org/techniques/T1140/.

[12] Mandiant/FireEye. "FLOSS - FireEye Labs Obfuscated String Solver."

GitHub: mandiant/flare-floss.

[13] Sebastiaan Marinus van Wiltenburg (2020). "Emulation-based deobfuscation of protected binaries." MSc Thesis, TU Delft.

[14] Google Cloud Security Blog (2023). "Automatically extracting obfuscated strings." https://cloud.google.com/blog/topics/threat-intelligence/automatically-extracting-obfuscated-strings/.

[15] Viettel Cyber Security Blog (2022). "Phương pháp deobfuscate trong quá trình phân tích mã độc Android." https://blog.viettelcybersecurity.com/.